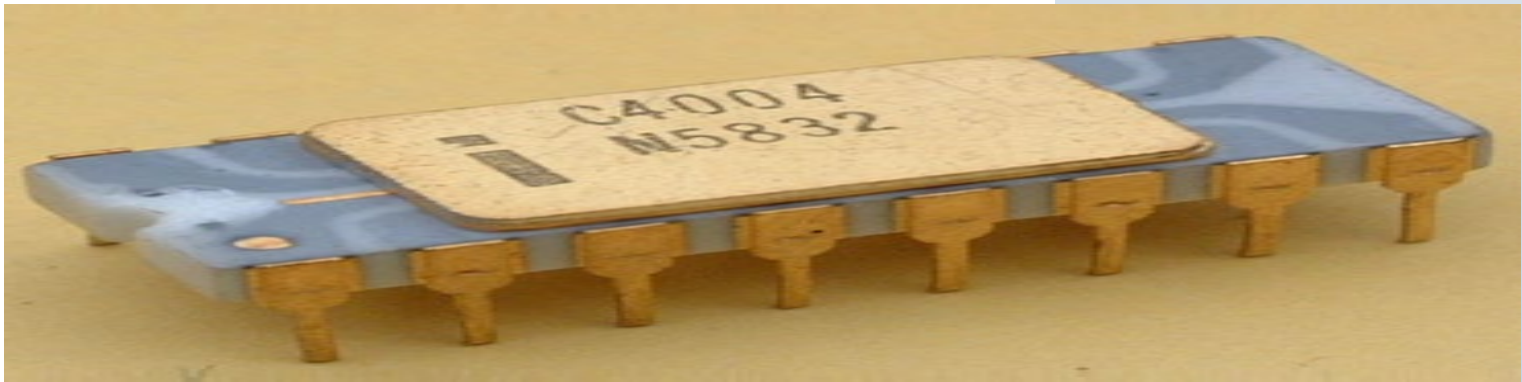




Le microprocesseur et son environnement





Objectifs de ce module

En suivant ce module vous allez :



- **Apprendre** le fonctionnement interne d'un CPU.
- **Comprendre** les interactions entre les différentes parties d'un CPU.
- **Découvrir** la manière dont sont utilisés les montages électroniques de base.





Plan du module

Voici les parties que nous allons aborder :



- **La construction de l'ALU et des registres internes.**
- **La gestion du temps.**
- **Le câblage du jeu d'instructions et la connexion avec l'extérieur.**
- **Le fonctionnement du microprocesseur.**





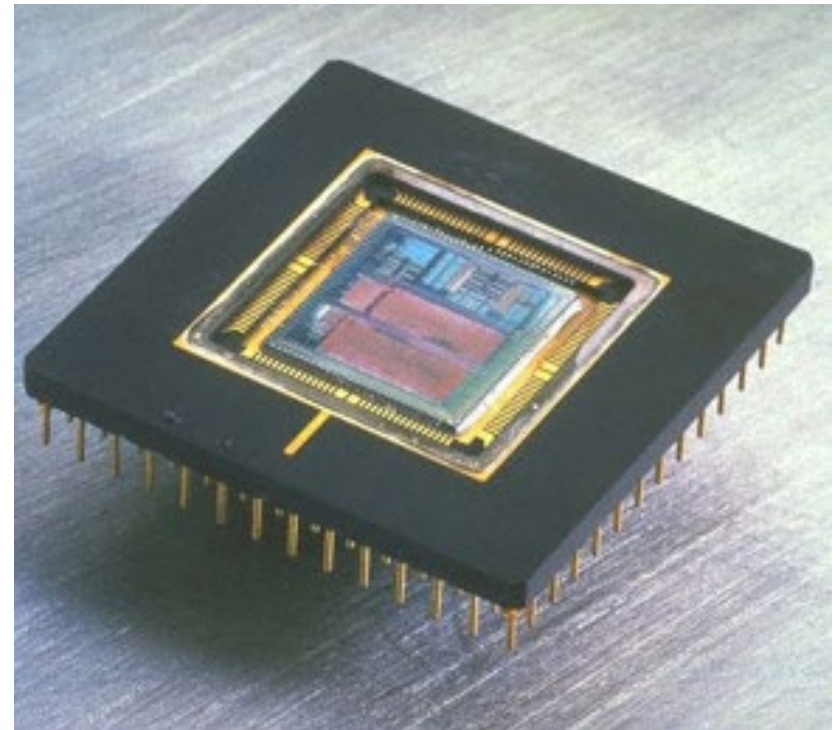
La construction de l'ALU et des registres internes



Plan de la partie

Voici les chapitres que nous allons aborder :

- Mise en place de l'ALU.
- Sélection des opérations.
- Mise en place des registres de données.
- Sélection des registres de données.
- Interconnexion registres internes – ALU.
- Le registre d'état.
- Le bus de contrôle et le signal RESET.
- Test du montage.





Mise en place de l'ALU

Objectif

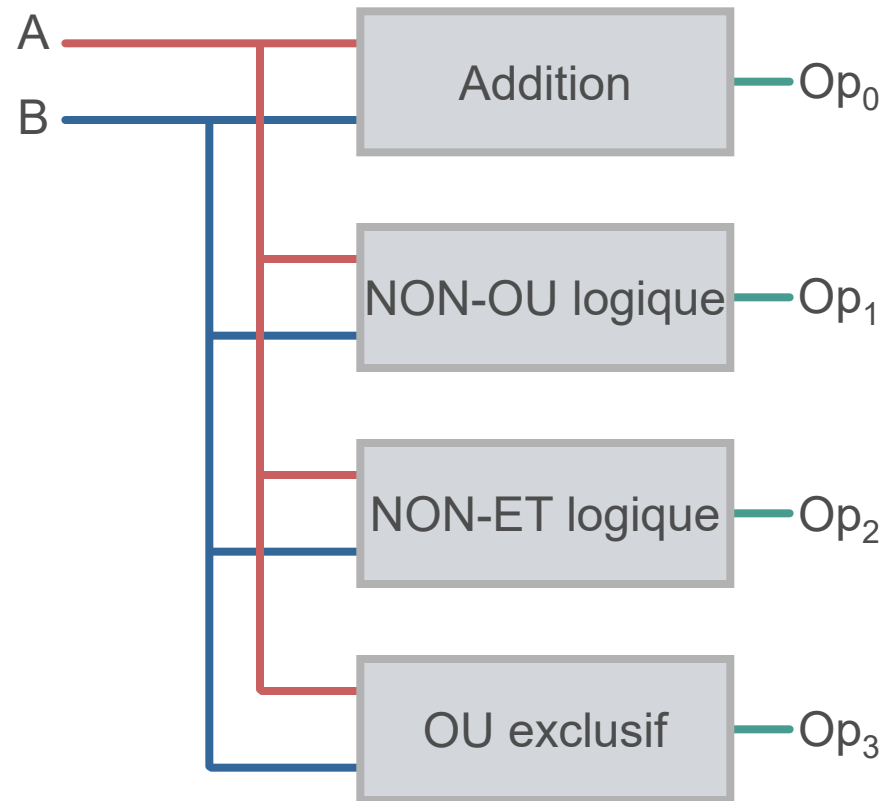
- Nous allons utiliser DigSim et construire une ALU 1 bit qui comportera 4 type d'opérations :
 - l'addition ;
 - le NON-OU logique ;
 - le NON-ET logique ;
 - le OU exclusif.
- Ces quatre montages doivent donc être placés de manière à avoir leur deux entrées (notée A et B) **communes** et leur sortie (notée Op_0 pour l'addition, Op_1 pour le NON-OU logique, Op_2 pour le NON-ET logique et Op_3 pour le OU exclusif) distinctes.





Mise en place de l'ALU

Schéma bloc du montage





Sélection des opérations

Objectif

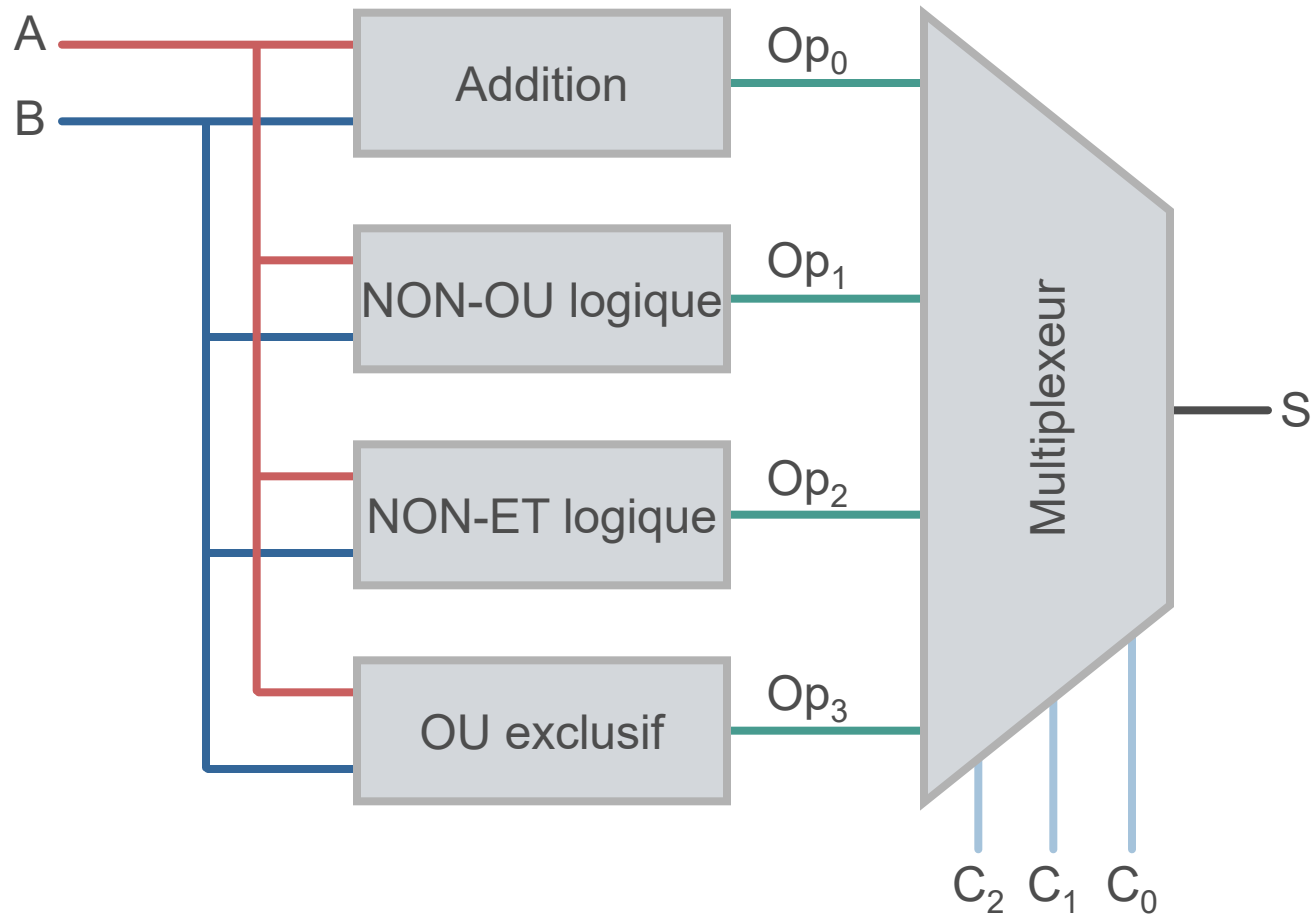
- Les 4 opérateurs de l'ALU fonctionnent simultanément, cependant à un instant donné, un seul résultat nous intéresse.
- Pour sélectionner le bon résultat, nous devons brancher un des quatre signaux Op_i à la sortie S de notre ALU : nous utilisons donc un **multiplexeur**.
- Les signaux de contrôle de ce multiplexeur seront notés C_0 et C_1 (nous avons 4 possibilités donc 2 bits de contrôle) et C_2 (un signal qui permet de contrôler l'état « branché ou débranché » de la sortie S du multiplexeur).





Sélection des opérations

Schéma bloc du montage





Mise en place des registres de données

Objectif

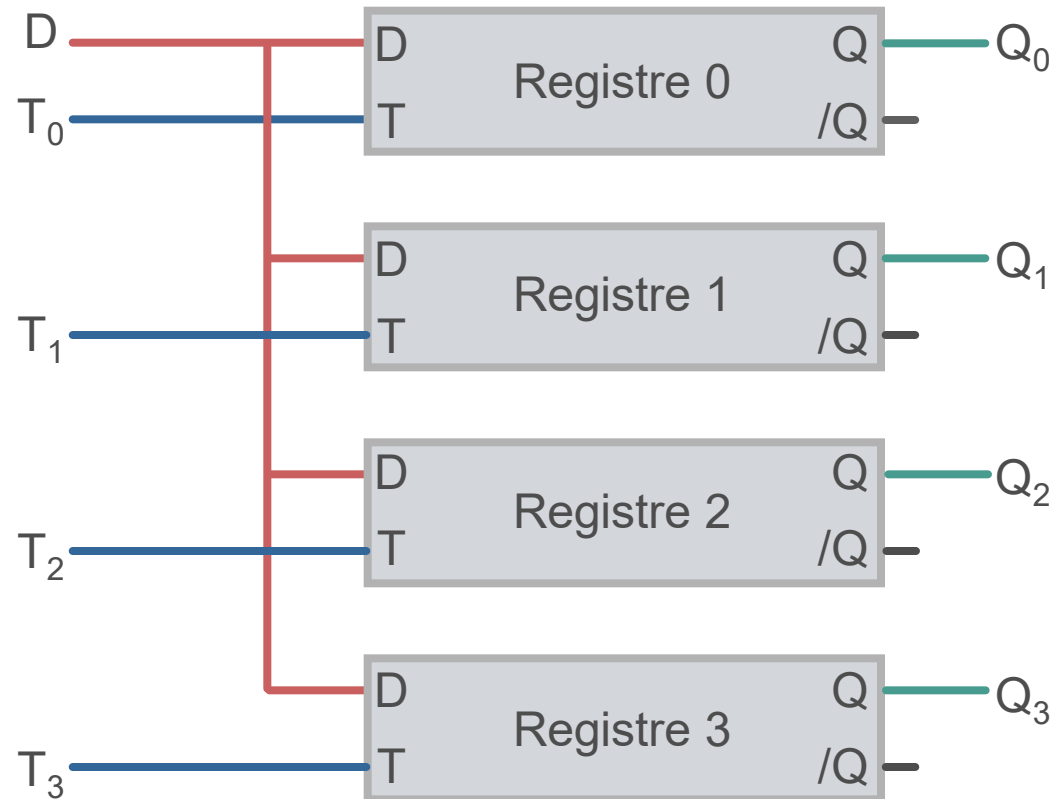
- A côté de ce premier montage, nous allons placer 4 registres de données 1 bit. Pour cela, nous utiliserons simplement 4 bascules D.
- L'interconnexion de ces bascules s'effectuera de la manière suivante :
 - l'entrée D de ces 4 bascules sera commune ;
 - les entrées (notées T_0 , T_1 , T_2 et T_3) seront distinctes ;
 - les sorties (notées Q_0 , Q_1 , Q_2 et Q_3) seront distinctes.
 - les sorties /Q sont inutilisées.





Mise en place des registres de données

Schéma bloc du montage





Sélection des registres de données

Objectif

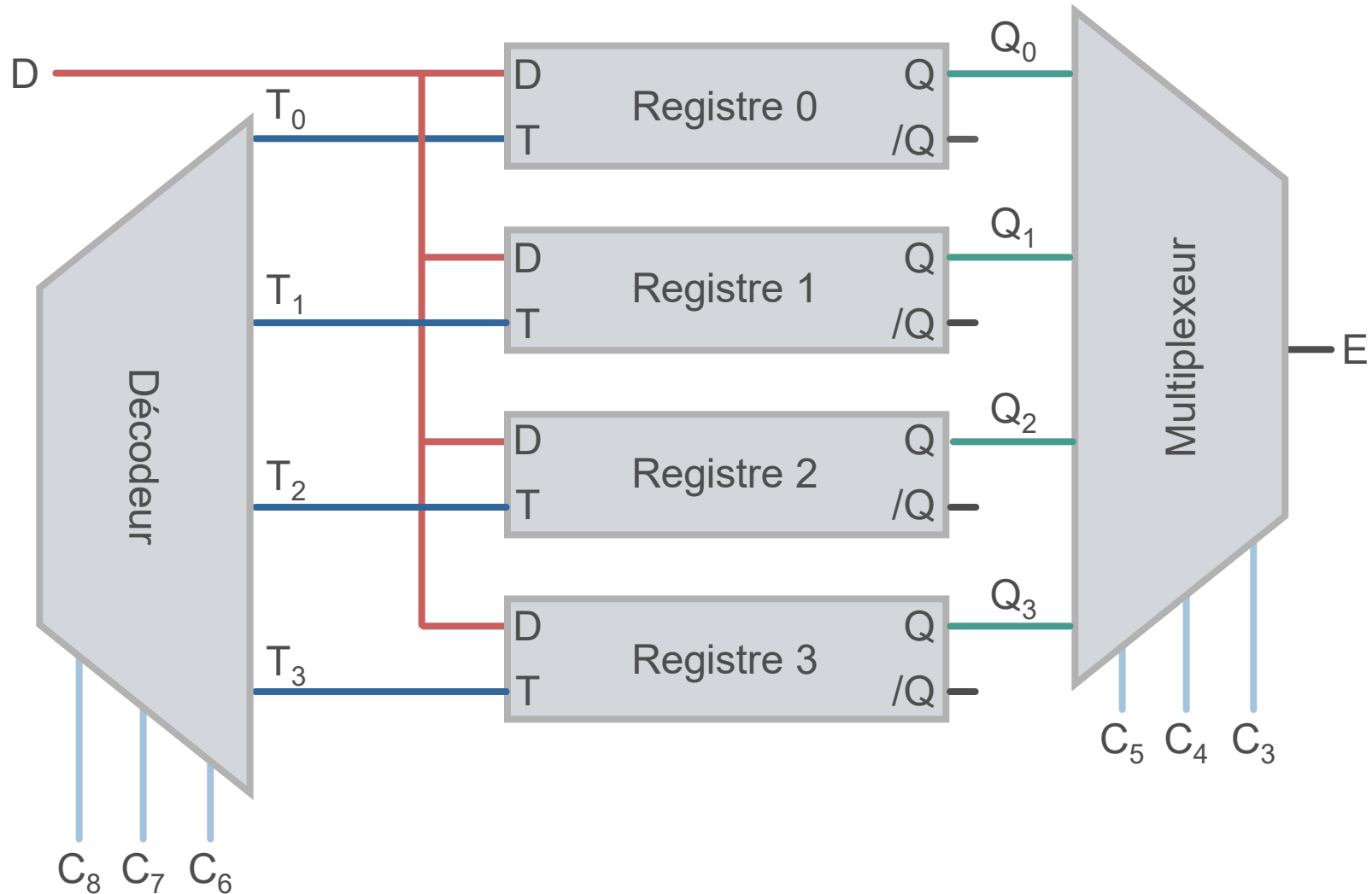
- Comme précédemment les 4 registres du processeur ont une entrée commune et des sorties distinctes :
 - Pour stocker l'information dans le bon registre, il suffit d'activer le bon T_i . Comme nous avons à chaque fois un T_i à sélectionner parmi les 4, nous utiliserons un **décodeur 2 vers 4**.
 - Pour récupérer l'information du bon registre, il faut de nouveau utiliser un **multiplexeur** qui va prendre en entrée les signaux Q_i et rendra en sortie un signal E .
- Les signaux de contrôle du décodeur seront notés C_3 , C_4 et C_5 et ceux du multiplexeur seront appelés C_6 , C_7 et C_8 .





Sélection des registres de données

Schéma bloc du montage





Interconnexion ALU – registres internes

Objectif

- Nous disposons maintenant de deux montages l'ALU et le bloc de registres internes.
- Nous allons les raccorder en mettant en place **un bus de données** qui va être commun aux entrées et aux sorties de l'ALU et du bloc de registres.
- Dans notre cas de figure, nous devrions relier les pattes A, B, D, E et S de notre montage par un même fil qui représente donc **un bus d'une largeur de 1 bit**.





Interconnexion ALU – registres internes

Problème

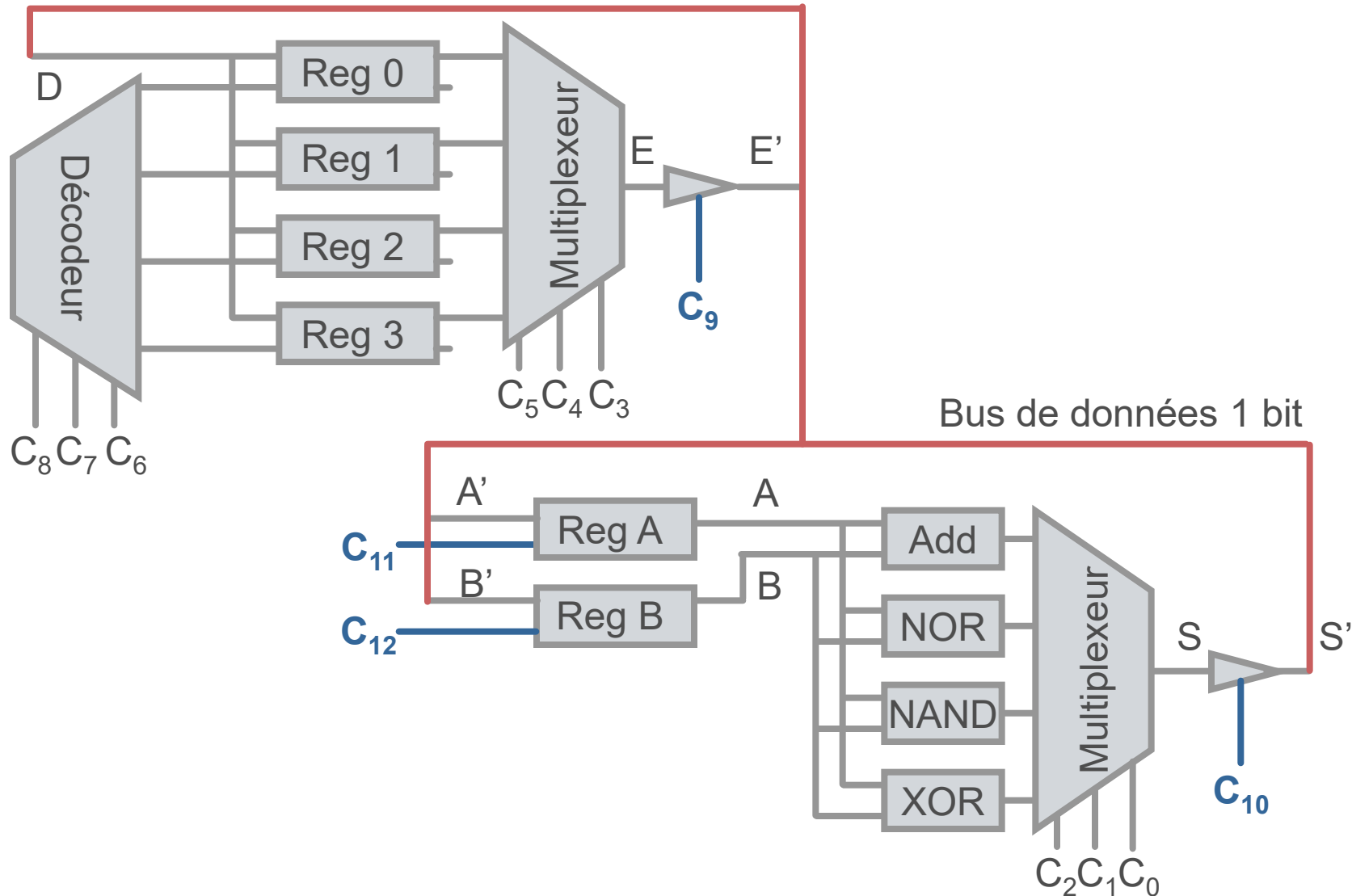
- Relier directement ces pattes entre elles risque de poser des problèmes électriques. Nous devons ajouter :
 - une **porte 3 états** à la sortie de notre bloc de registres (juste après la patte E, nous obtenons une patte E') ;
 - une autre **porte 3 états** à la sortie de l'ALU (juste après la patte S, nous obtenons donc une patte S') ;
 - deux registres qui permettront de stocker de façon temporaire les opérandes A et B (à placer juste avant les pattes A et B) : nous obtenons les pattes A' et B'.
- Nous pouvons donc relier les pattes A', B', S', E' et D entre elles par un fil et nous notons C_9 , C_{10} , C_{11} et C_{12} les nouvelles pattes de contrôle de ce montage.





Interconnexion ALU – registres internes

Schéma bloc du montage





Le registre d'état

Objectif

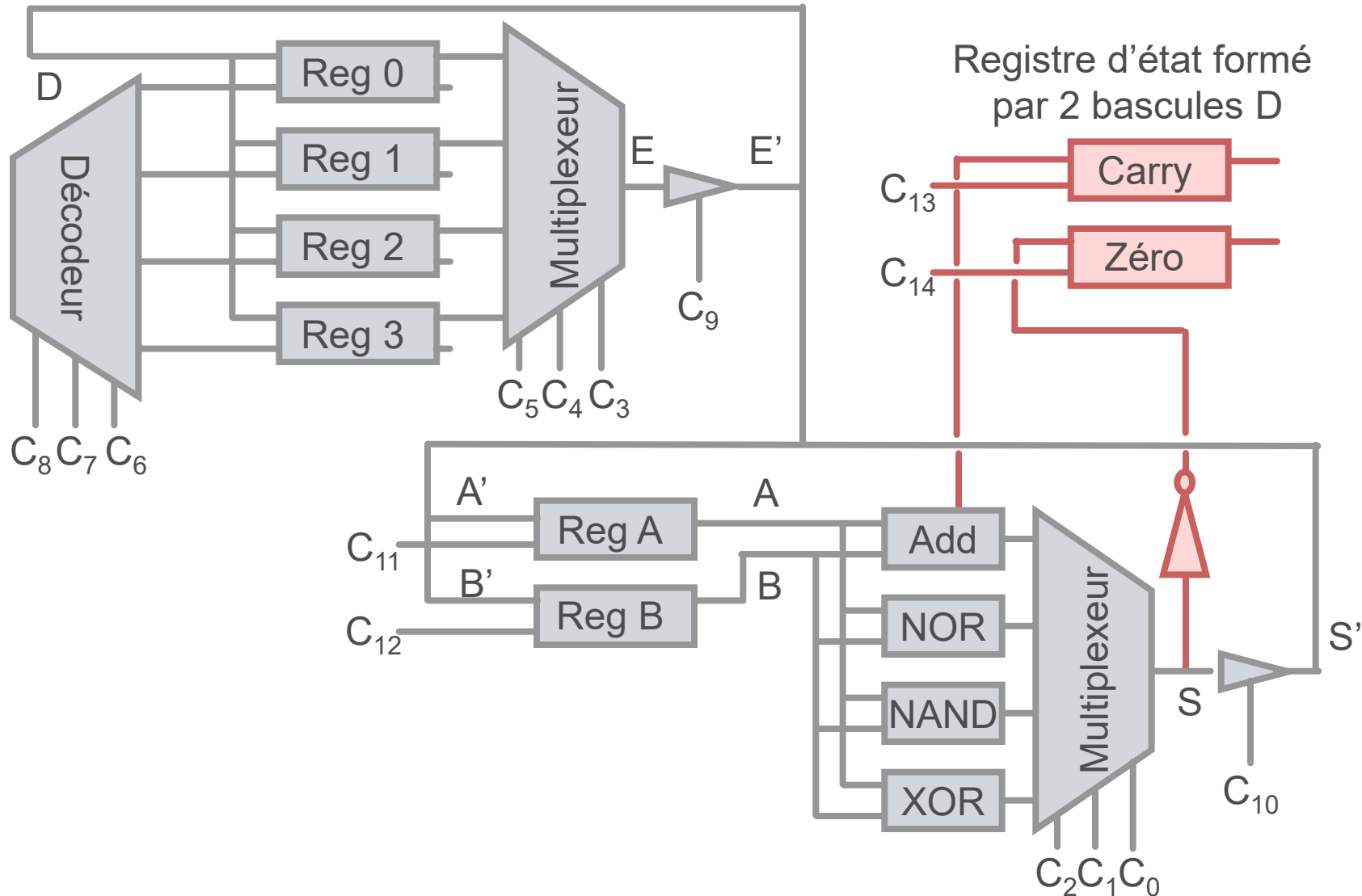
- Notre additionneur rend un résultat sur 2 bits :
 - le bit issu de l'addition des deux bits placés en entrée ;
 - le bit de retenu que nous n'avons pas encore utilisé.
- Nous allons placer deux nouvelles bascules D qui permettront de **stocker l'état de notre ALU** lors de chaque opération :
 - si l'addition provoque une retenue, le premier registre doit contenir 1 (on appellera cette information « le bit carry »)
 - si une opération retourne un zéro, le second registre doit contenir 1 (on appellera cette information « le bit zéro »).





Le registre d'état

Schéma bloc du montage





Le bus de contrôle et le signal reset

Objectif

- Tous les signaux C_i mis ensemble forment le **bus de contrôle** de notre processeur 1 bit.
- Dans un premier, nous allons ajouter un certain nombre de commutateurs afin de contrôler l'état des signaux C_i .
- Par la suite ces signaux seront câblés à d'autres éléments du processeurs comme le **séquenceur** et le **décodeur d'instructions**.





Le bus de contrôle et le signal reset

Objectif

- Pour faciliter la manipulation de notre processeur, nous devons ajouter un signal supplémentaire appelé **RESET** (qui sera commandé par un commutateur) :
 - si le signal est à 1, cela force à enregistrer un « 0 » dans toutes les bascules D du montage ;
 - si le signal est à 0, le montage fonctionne normalement.
- Déterminer comment ajouter un tel signal dans le montage.





Test du montage

Objectif

- Maintenant que nous disposons d'un processeur 1 bit, déterminer la séquence à effectuer au niveau des commutateur C_i pour effectuer les suites d'instructions ci-dessous :
 - RESET ;
 - Faire un NON-OU entre Reg 0 et Reg 1 ;
 - Stocker le résultat dans Reg 2 ;
 - Additionner Reg 0 et Reg 2 ;
 - Stocker le résultat dans Reg 3 ;
 - Additionner Reg 2 et Reg 3 ;
 - Stocker le résultat dans Reg 0.





Pause-réflexion sur cette 1^{ère} partie

Avez-vous des questions ?





La gestion du temps



Plan de la partie

Voici les chapitres que nous allons aborder :

- Le quartz.
- L'horloge à quartz.
- Le séquenceur du processeur.





L'horloge à quartz

Le signal d'horloge « de base »

- L'horloge est un système qui émet **régulièrement** une suite d'impulsions calibrées permettant de synchroniser le fonctionnement des éléments d'un ordinateur.
- L'horloge est caractérisée par son **temps de cycle** ou sa **période d'horloge**.
- Selon la manière dont est taillé le quartz, on obtient des fréquences allant jusqu'à 100 ou 150 Mhz.





Pause-réflexion sur cette 1^{ère} partie

Avez-vous des questions ?





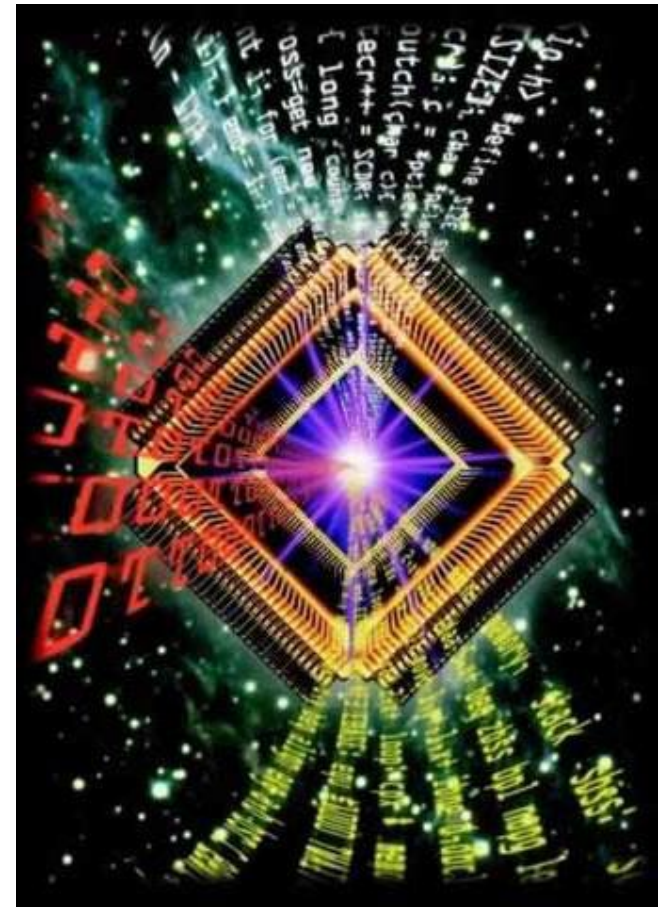
Le câblage du jeu d'instructions et la connexion avec l'extérieur



Plan de la partie

Voici les chapitres que nous allons aborder :

- La notion de jeu d'instructions.
- Le décodeur d'instructions.
- La place de la mémoire externe.
- L'adressage de la mémoire.
- La connexion avec l'exterieur





La notion de jeu d'instructions

Les défauts de notre premier CPU

- Le CPU que nous avons construit jusqu'à présent présente deux gros défauts :
 - Il ne sait faire qu'une seule suite d'instructions que nous avons « écrit en dur dans le processeur » en connectant les pattes C_i directement au séquenceur.
 - Ce processeur ne traite aucune donnée externe et il ne renvoie aucun résultat (il ne prend également aucune instruction du monde extérieur).





La notion de jeu d'instructions

Construction du jeu d'instructions – Exemple de la partie 1

- Reprenons la suite d'instructions que nous avons utilisées pour tester notre premier circuits :
 - faire un NON-OU entre Reg 0 et Reg 1 ;
 - stocker le résultat dans Reg 2 ;
 - additionner Reg 0 et Reg 2 ;
 - stocker le résultat dans Reg 3 ;
 - additionner Reg 2 et Reg 3 ;
 - stocker le résultat dans Reg 0.





La notion de jeu d'instructions

Construction du jeu d'instructions – Exemple de la partie 1

- Les deux premières instructions peuvent être considérées comme une seule instruction évoluée :
 - faire un NON-OU entre Reg 0 et Reg 1 ;
 - stocker le résultat dans Reg 2.

- Nous avons donc l'instruction : **$\text{Reg 2} \leftarrow \neg(\text{Reg 0} + \text{Reg 1})$**





La notion de jeu d'instructions

Construction du jeu d'instructions – Les variantes

- Cette instruction évoluée peut être déclinée en :
 - $\text{Reg } 0 \leftarrow /(\text{Reg } 0 + \text{Reg } 1)$
 - $\text{Reg } 1 \leftarrow /(\text{Reg } 0 + \text{Reg } 1) \dots$
 - $\text{Reg } 2 \leftarrow /(\text{Reg } 3 + \text{Reg } 0) \dots$

- Toutes ces instructions évoluées ont toutes un fonctionnement proche (il s'agit du même type de séquencement).





La notion de jeu d'instructions

Construction du jeu d'instructions – Les variantes

- Finalement nous obtenons un **ensemble d'instructions plus évoluées** qui permettent de contrôler plus facilement et plus finement le fonctionnement notre processeur :
 - nous sélectionnons l'instruction évoluée à exécuter ;
 - cela entraîne l'activation programmée des éléments du processeur qui vont permettre l'exécution de cette instruction.
- Cet ensemble d'instructions forme ce que nous appelons communément **le jeu d'instructions du processeur**.





Le décodeur d'instructions

Présentation

- Pour faire le lien entre ces instructions évoluées et le séquençement des opérations de base, nous devons ajouter un circuit supplémentaire appelé **décodeur d'instructions** qui comportera :
 - des lignes d'entrée afin de recevoir le numéro (en binaire) de l'instruction à effectuer ;
 - des lignes de sortie qui permettront de **pré-activer** les éléments nécessaires à la réalisation de notre instruction.





Le décodeur d'instructions

Connexion avec le séquenceur

- « Pré-activer » signifie que cette activation sera effective lorsque l'élément concerné sera « désigné » par le séquenceur.
- Autrement dit, un élément du CPU sera **activé** lorsqu'il aura été préalablement **pré-activé** par le décodeur **et** qu'il sera **désigné** par le séquenceur.
- Le signal permettant d'activer un élément sera donc issu d'une **porte ET** à deux entrées :
 - la première entrée provient du séquenceur ;
 - la seconde entrée est issue du décodeur d'instructions.

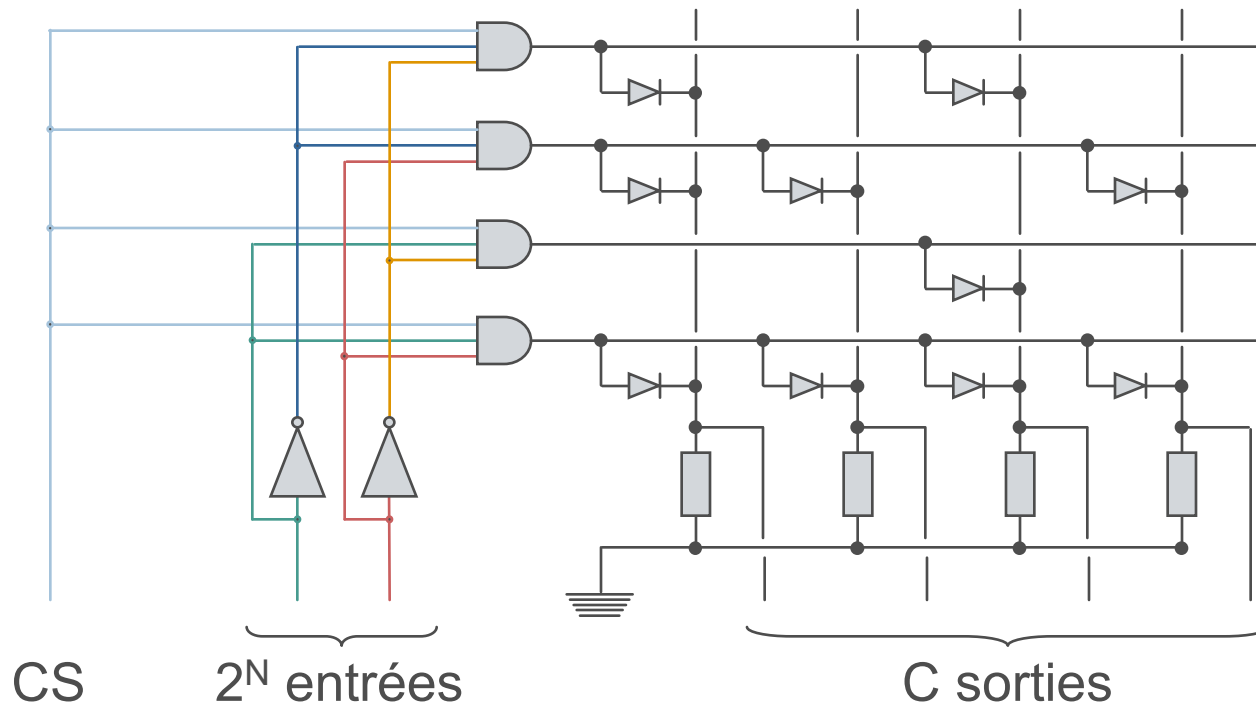




Le décodeur d'instructions

Construction

- Le décodeur d'instructions est en fait une simple ROM possédant 2^N entrées (les lignes adresses) et C sorties pour établir une correspondance entre l'entrée (l'instruction évoluée) et les lignes C_i à pré-activer.





Le décodeur d'instructions

Objectif

- Construire le décodeur d'instructions afin d'implanter le jeu d'instructions ci-dessous :
 - 0 : $\text{Reg } 0 \leftarrow \text{Reg } 0 + \text{Reg } 1$
 - 1 : $\text{Reg } 0 \leftarrow /(\text{Reg } 0 + \text{Reg } 1)$
 - 2 : $\text{Reg } 0 \leftarrow /(\text{Reg } 0 . \text{Reg } 1)$
 - 3 : $\text{Reg } 0 \leftarrow \text{Reg } 0 \oplus \text{Reg } 1$
 - 4 : $\text{Reg } 1 \leftarrow \text{Reg } 0 + \text{Reg } 1$
 - 5 : $\text{Reg } 1 \leftarrow /(\text{Reg } 0 + \text{Reg } 1)$
 - 6 : $\text{Reg } 1 \leftarrow /(\text{Reg } 0 . \text{Reg } 1)$
 - 7 : $\text{Reg } 1 \leftarrow \text{Reg } 0 \oplus \text{Reg } 1 \quad \dots \text{ etc}$

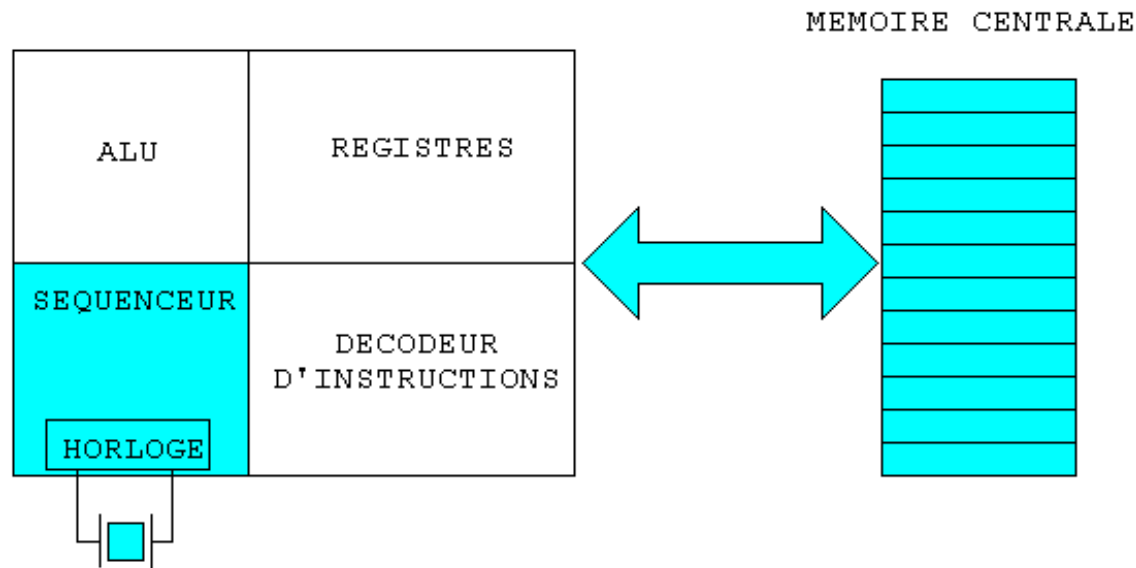




La place de la mémoire externe

L'intérêt de la mémoire

- Pour fonctionner tout seul, un microprocesseur a besoin qu'on lui fournisse de manière automatique les instructions et les données à traiter.
- Pour cela, nous allons utiliser **une mémoire externe** (par opposition aux registres qui sont internes aux microprocesseur).

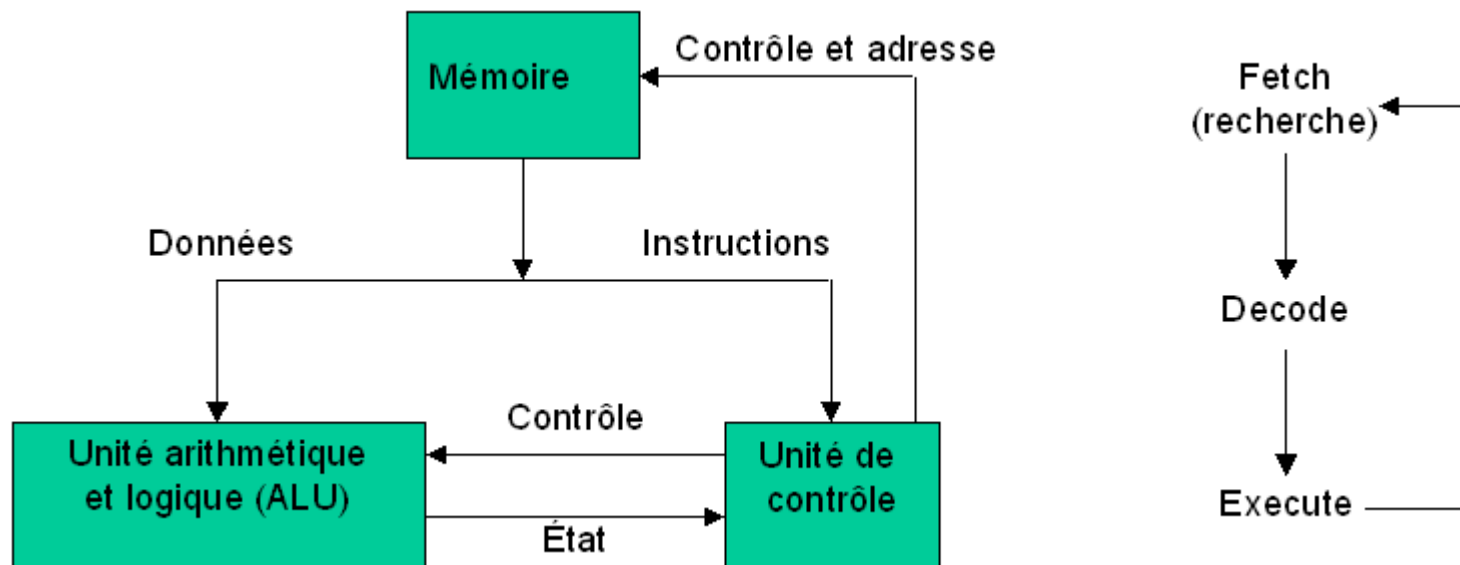




La place de la mémoire externe

L'architecture de Von Neumann

- Dans cette organisation, la mémoire et les registres servent à la fois à stocker des instructions et des données. Ce type d'organisation, utilisé dans la grande majorité des ordinateurs, est appelée **architecture de von Neumann**.
- Son fonctionnement se déroule en 3 grandes étapes





L'adressage de la mémoire

Le compteur ordinal

- Pour accéder de façon automatique à une case mémoire, nous allons devoir utiliser un **compteur** .
- Ce compteur sera considéré comme un registre spécial : le **compteur ordinal**.
- La valeur de ce compteur va désigner **la case mémoire** contenant la prochaine instruction à charger : cela correspond à l'**adresse** de la prochaine instruction.





L'adressage de la mémoire

Les registres d'adresse

- Le compteur ordinal contient une adresse, mais le processeur contient généralement d'autres registres qui permettent de stocker d'autres adresses utilisées au sein de notre programme assembleur.
- Ces registre sont appelées **registres d'adresse**.
- Nous reviendrons sur leur utilisation (leur raison d'être) lorsque nous détaillerons l'organisation du 8086 en vue de sa programmation.





L'adressage de la mémoire

Le bus d'adresse

- Nous allons positionner un compteur et 3 registres qui fonctionnent sur 8 bits afin de pouvoir adresser un espace mémoire de 2^8 bits.
- Câbler le circuit de manière à ce que le compteur soit incrémenté à chaque début du cycle de fonctionnement de votre processeur.
- Ajouter le bus d'adresse afin d'interconnecter les sorties et les entrées de ces 4 éléments entre eux (ajouter les composants nécessaires en s'inspirant des registres de données).





L'adressage de la mémoire

Le bus d'adresse

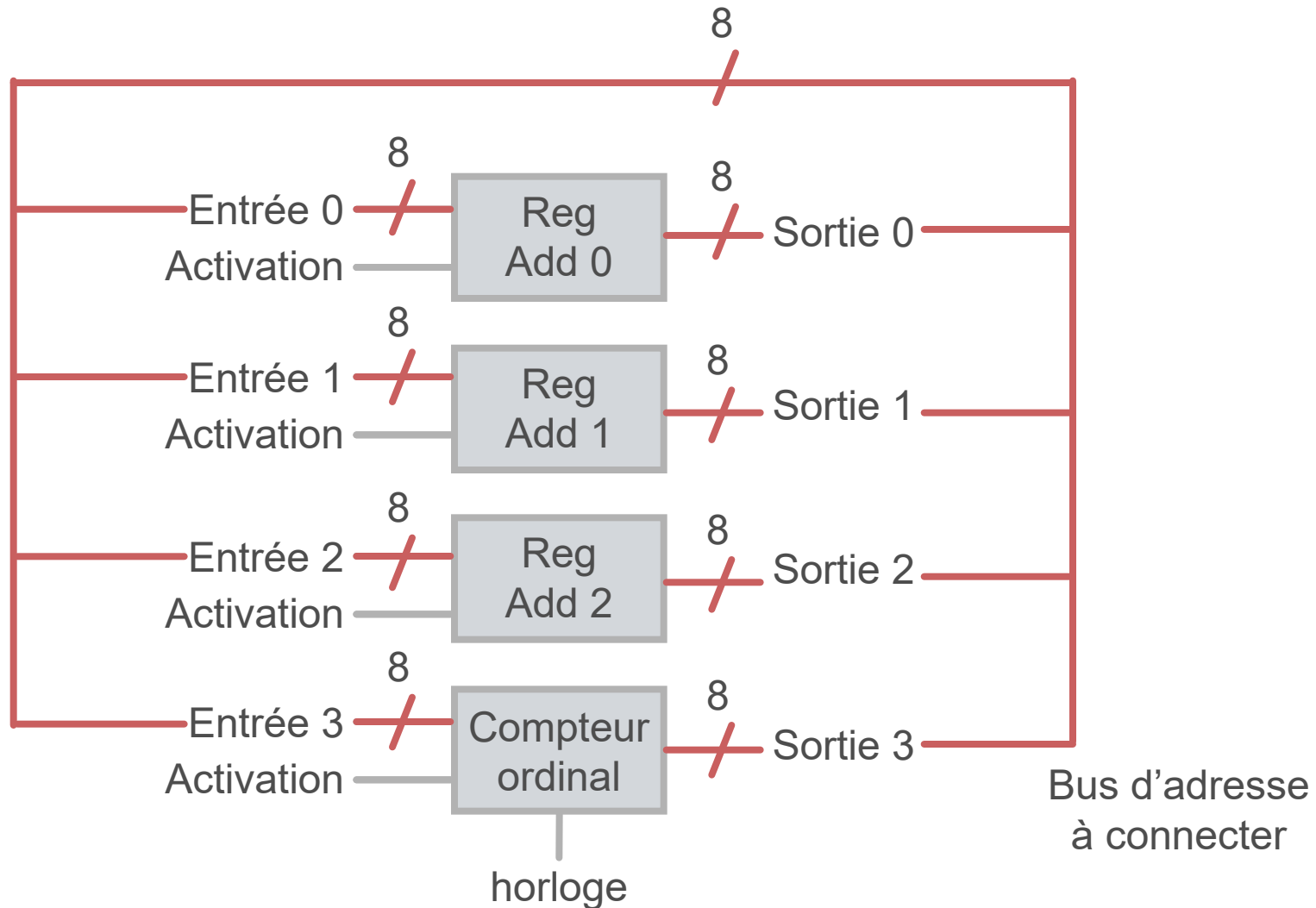
- Remarque 1 : Le compteur ordinal peut être initialisé de façon logicielle à une valeur particulière donc il dispose d'une entrée comme les autres registres d'adresse.
- Remarque 2 : Le bus de données pourrait également être connecté aux registres d'adresse car il est possible (en assembleur) de transmettre des adresses d'un registre de données et inversement mais ce cas ne sera pas traité ici.





L'adressage de la mémoire

Le bus d'adresse

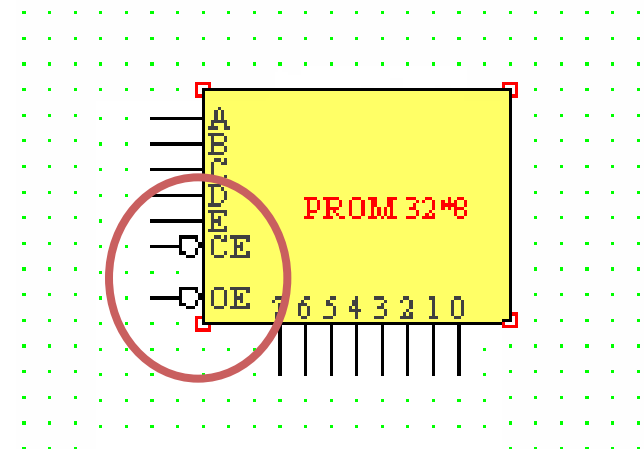
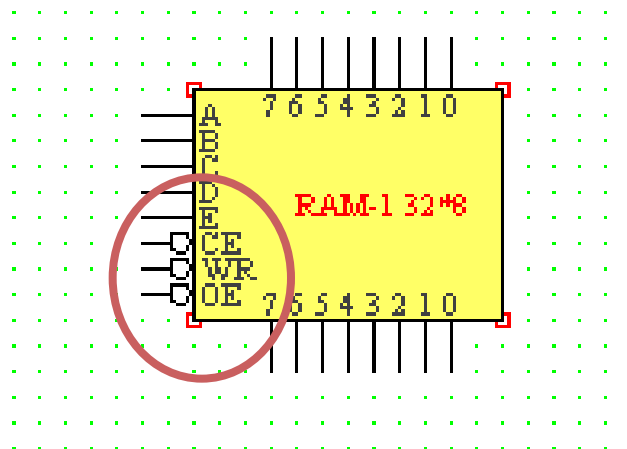




La connexion avec l'extérieur

Objectif

- Nous allons ajouter des blocs de mémoire (RAM et ROM) de manière à obtenir 2^8 cases mémoire.
- Ces mémoires nécessitent des signaux particuliers pour fonctionner. Ces signaux devront être contrôlés par le processeurs et ils viendront s'ajouter aux signaux de contrôle déjà existants (éléments supplémentaires à séquencer)





La connexion avec l'extérieur

Le bus d'adresse et le bus de données externes

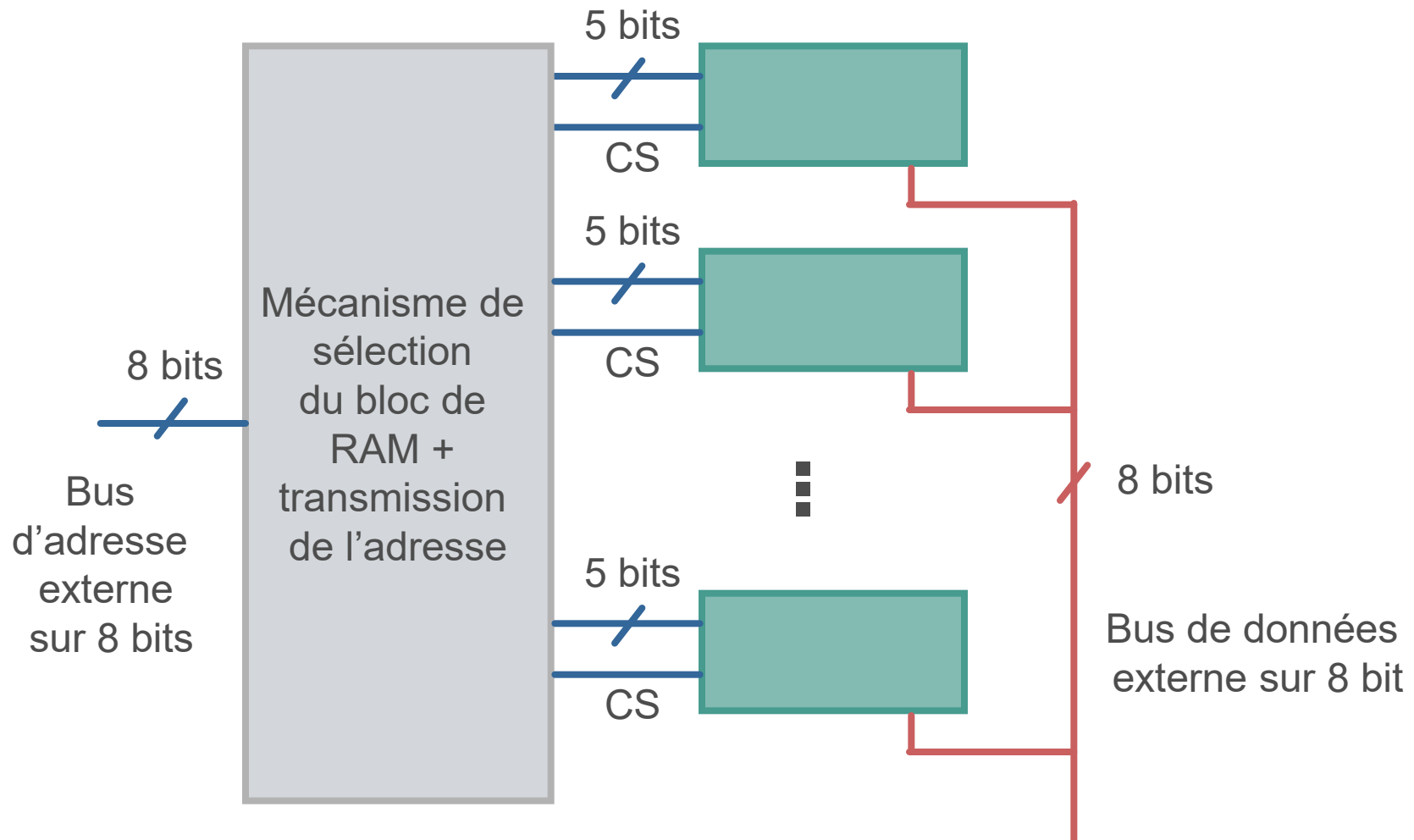
- Nous allons maintenant connecter les blocs de mémoire entre eux de manière à obtenir :
 - **8 lignes d'adresse** (les blocs de mémoire, que nous utilisons, comportent 5 lignes d'adresse, il faut mettre en place un mécanisme pour sélectionner le bon bloc de RAM en utilisant les lignes d'adresse restantes) ;
 - **8 lignes de données communes.**





La connexion avec l'extérieur

Le bus d'adresse et le bus de données externes





La connexion avec l'extérieur

La connexion entre les bus internes et externes

- Il ne nous reste alors plus qu'à ajouter des **buffers à 3 états** afin de connecter
 - le bus de données interne et le bus de données externe ;
 - le bus d'adresse interne et le bus d'adresse externe.
- Ce qui ajoute de nouveau signaux de contrôle qui devront être pris en compte dans le séquençement.





Le câblage du jeu d'instructions et la connexion avec l'extérieur

Pause-réflexion sur cette 3^{ème} partie

Avez-vous des questions ?





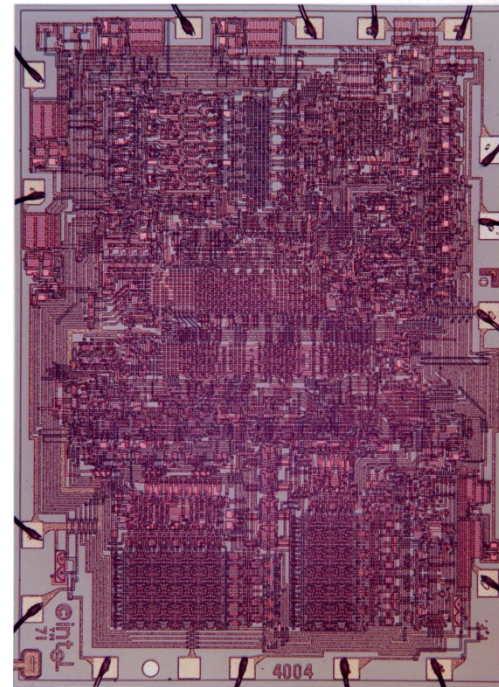
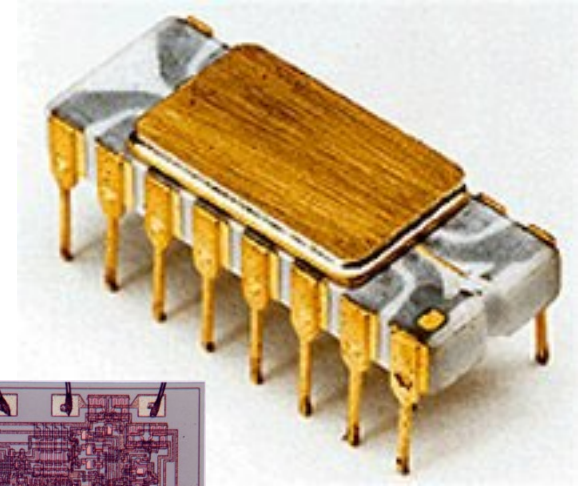
Le fonctionnement du microprocesseur



Plan de la partie

Voici les chapitres que nous allons aborder :

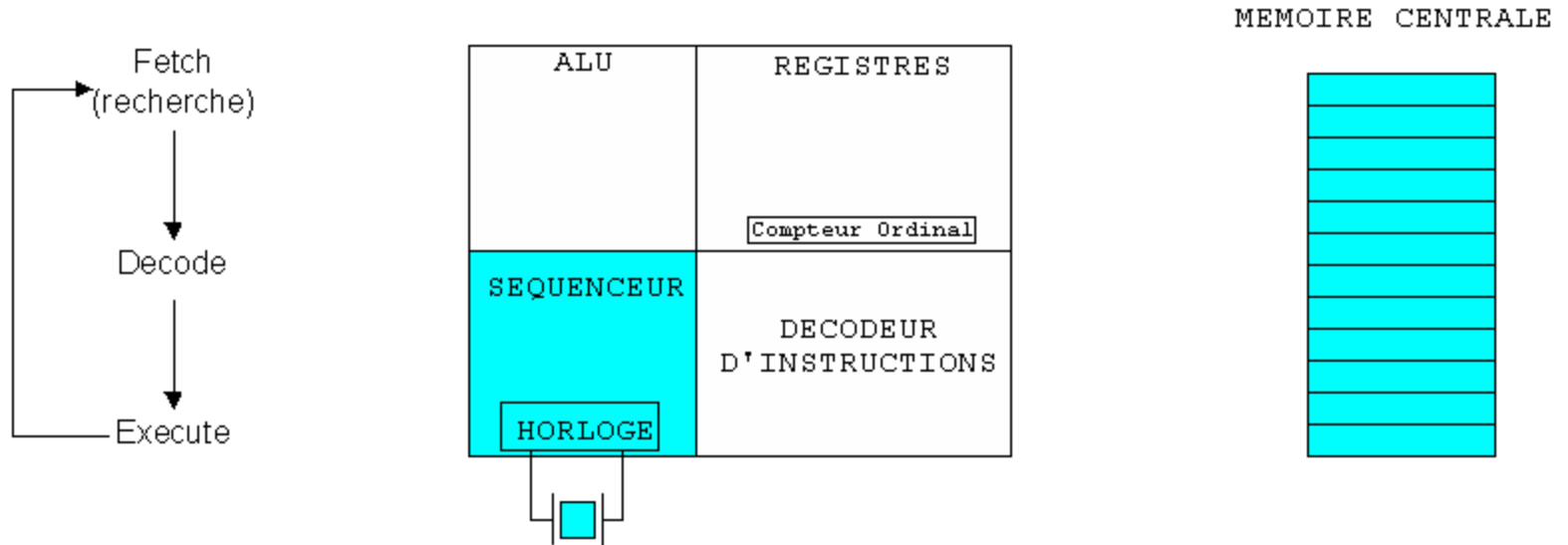
- Description du fonctionnement du processeur.
- Le cas du 4004.





Description du fonctionnement

Le CPU et la mémoire



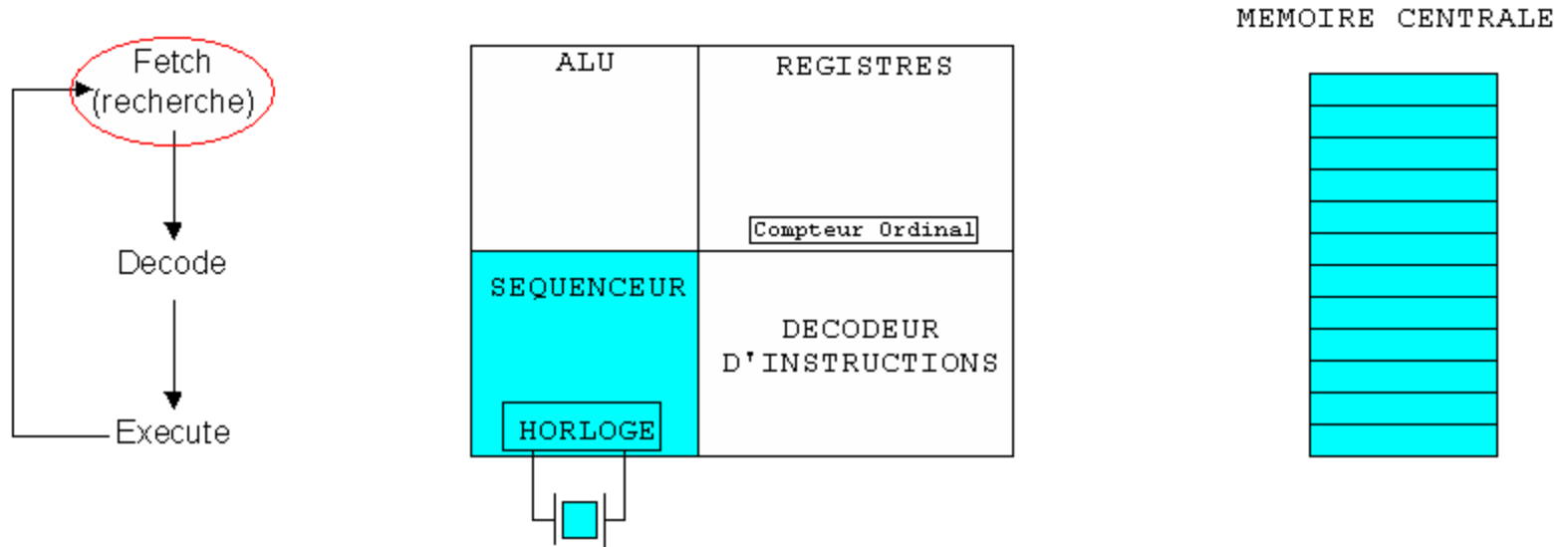
On dispose d'un CPU (respectant l'architecture de Von Neumann) et une mémoire centrale. Le processeur va entamer son cycle à 3 phases (recherche, décodage, exécution).





Description du fonctionnement

Phase de recherche



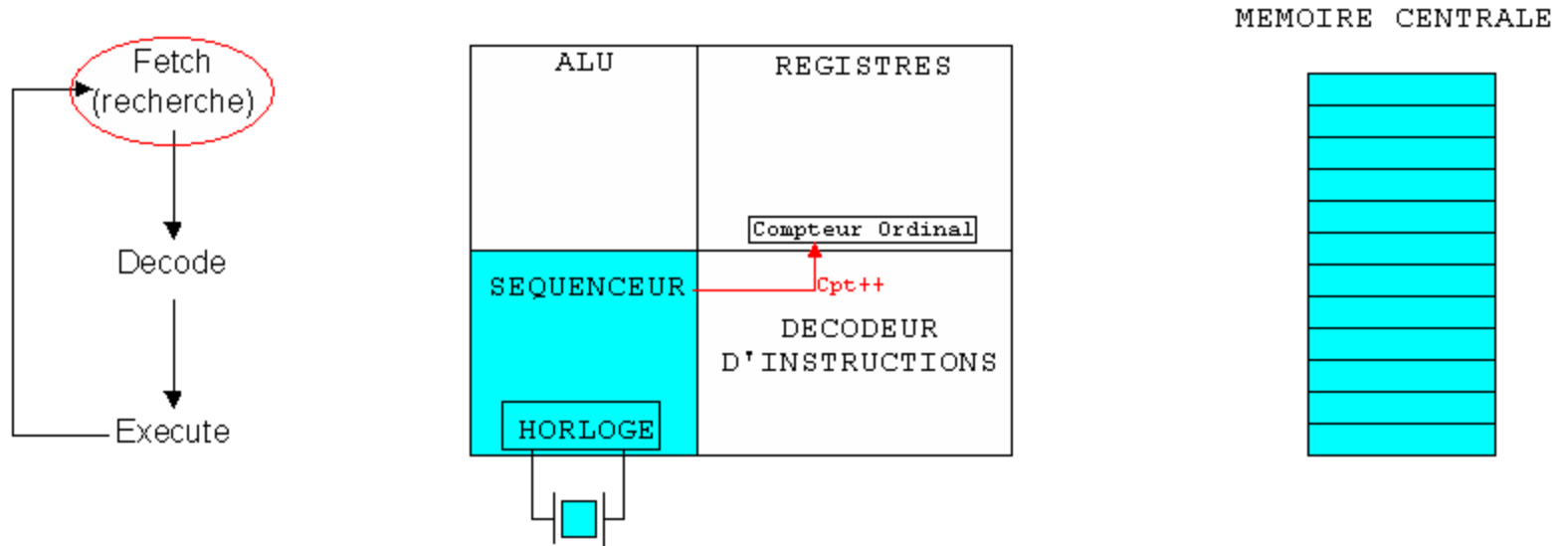
On commence la phase de recherche de l'instruction dans la mémoire centrale pour la charger dans le CPU afin qu'il puisse l'exécuter.





Description du fonctionnement

Phase de recherche – incrémentation du compteur ordinal



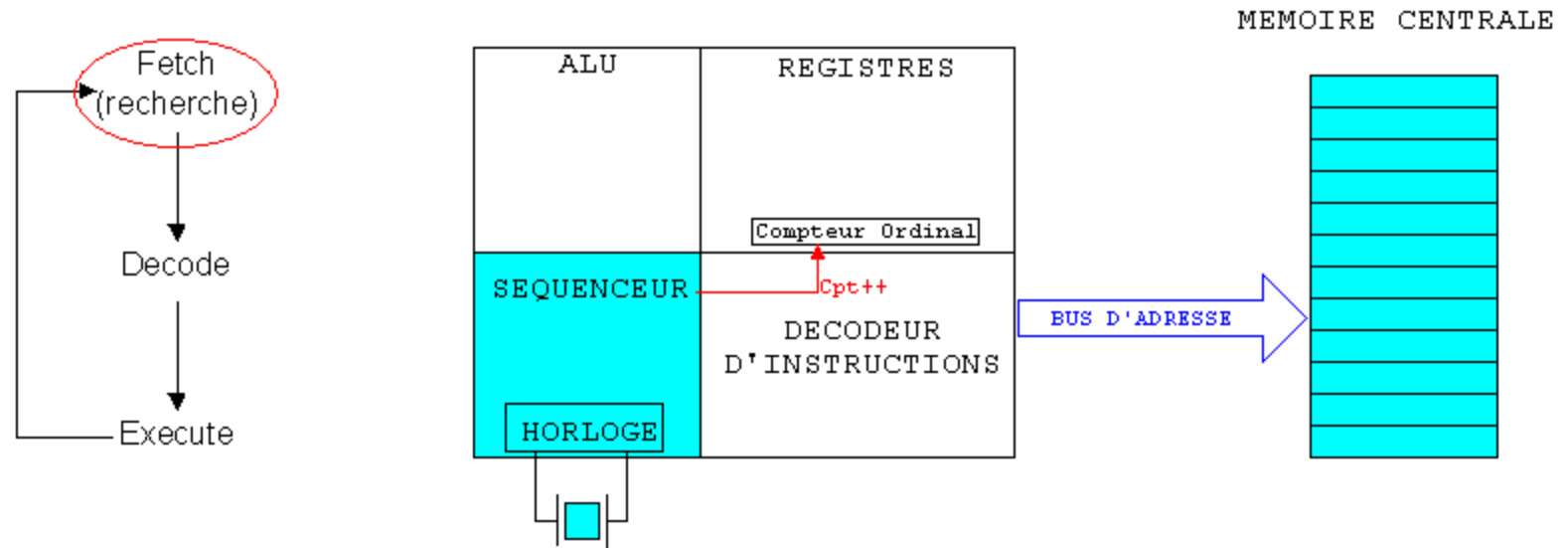
Le séquenceur incrémente le compteur ordinal, un registre qui stocke le numéro de la case mémoire (l'adresse) où se trouve la prochaine instruction à exécuter.





Description du fonctionnement

Phase de recherche – Envoi de l'adresse à la RAM



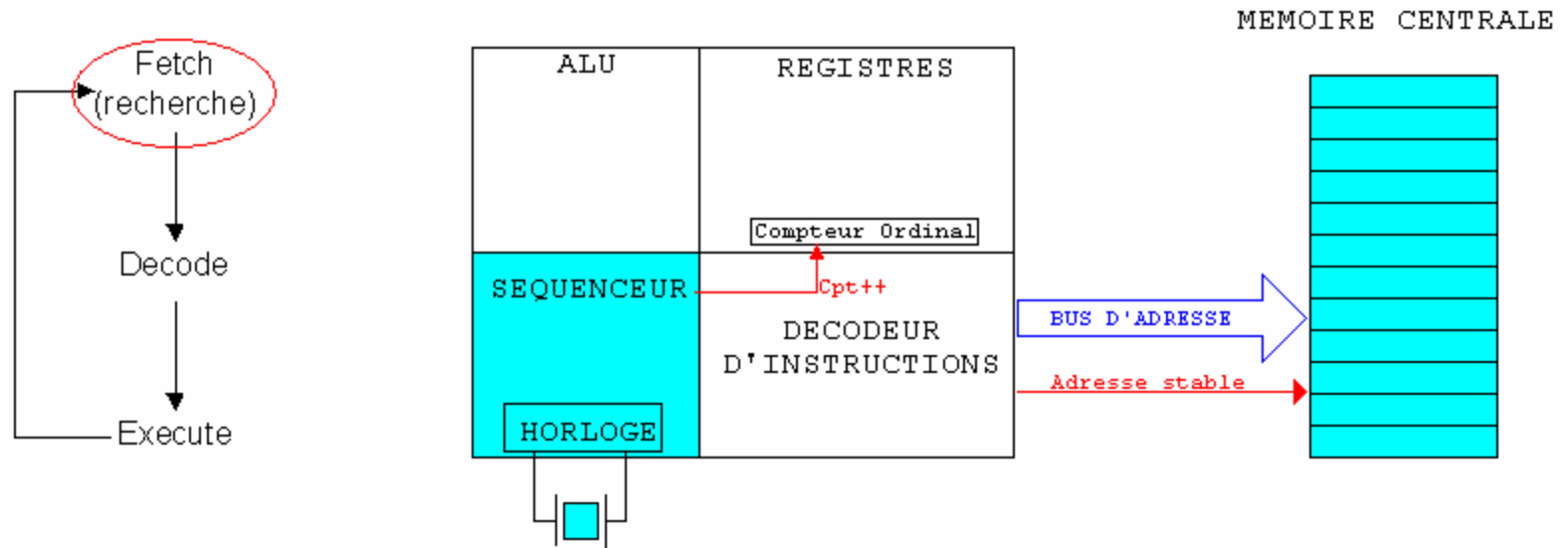
La valeur du compteur ordinal est envoyée sur le bus d'adresse, autrement dit si le compteur ordinal contient 01101101, le premier fil du bus a une tension de 0v, le second fil une tension de 5v ...etc





Description du fonctionnement

Phase de recherche – Envoi de l'adresse à la RAM



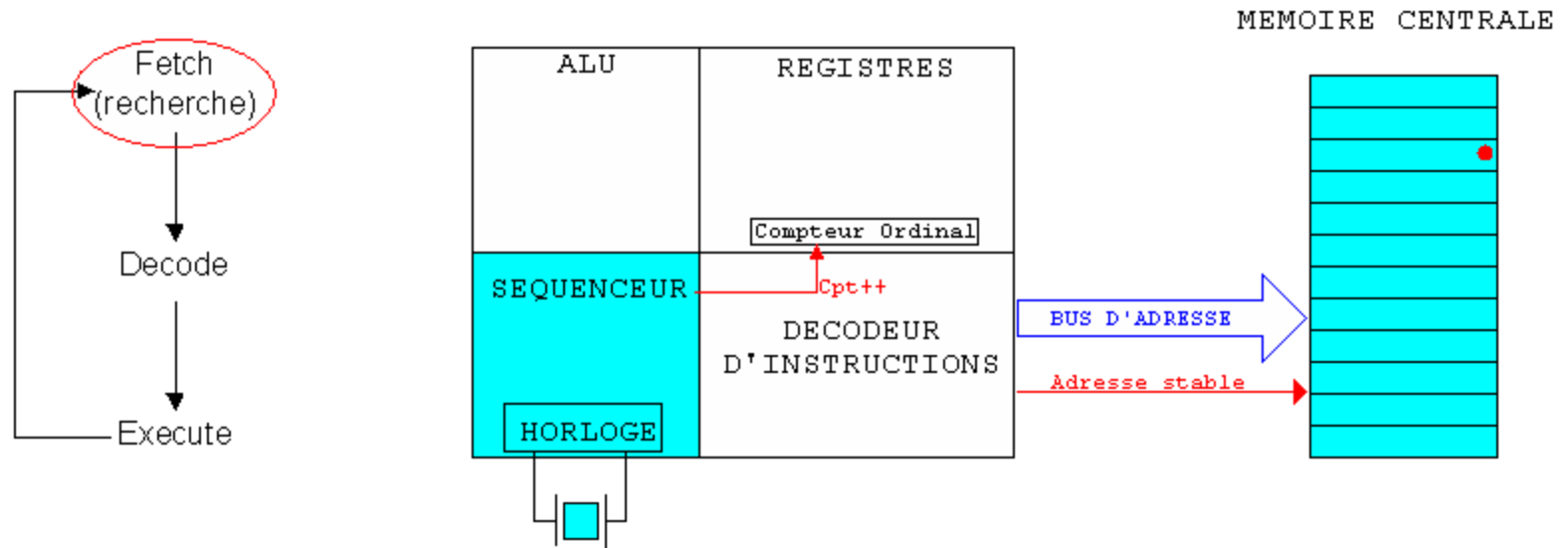
Lorsque l'état de tous les fils du bus d'adresse est stabilisé, le processeur envoie un signal (par un autre fil) vers la mémoire centrale pour lui indiquer qu'il peut lire le bus d'adresse.





Description du fonctionnement

Phase de recherche – Sélection de la case mémoire



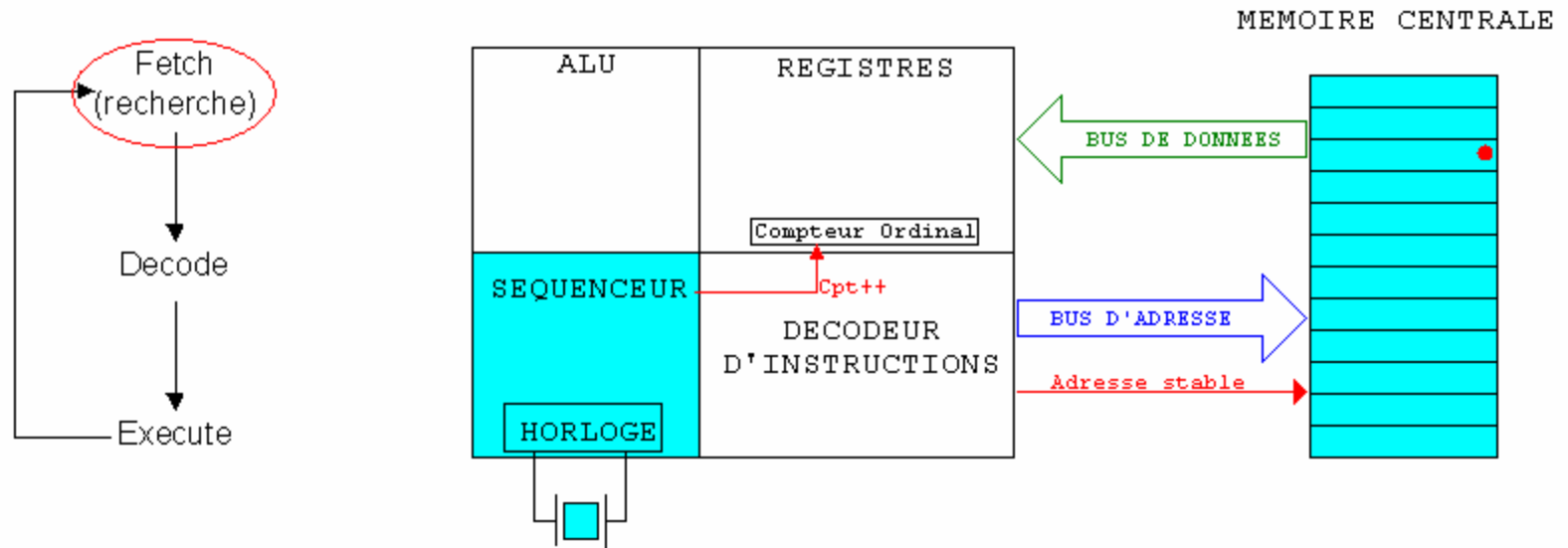
La mémoire sélectionne la case mémoire correspondante à la valeur placée sur le bus d'adresse.





Description du fonctionnement

Phase de recherche – Envoi des données vers le CPU



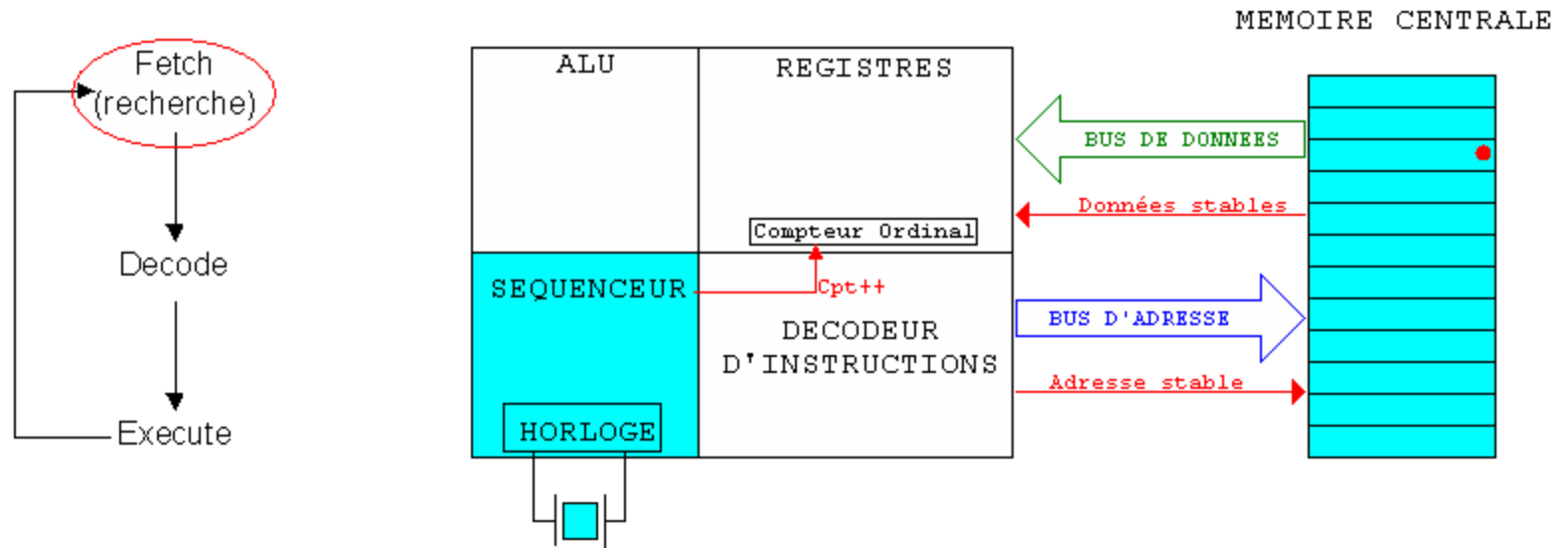
La mémoire place sur un autre bus (le bus de données), la valeur stockée dans la case de la mémoire centrale.





Description du fonctionnement

Phase de recherche – Envoi des données vers le CPU



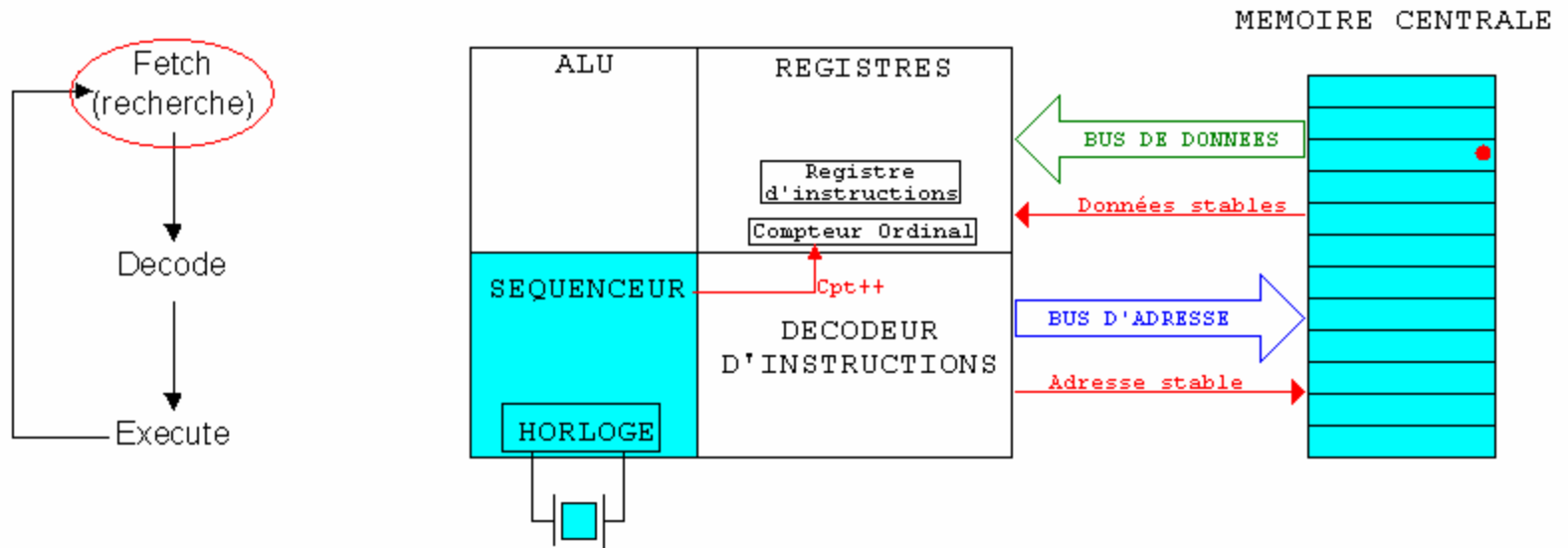
Lorsque les fils du bus de données ont un état stable, la mémoire envoie un signal pour indiquer au CPU qu'il peut lire la donnée.





Description du fonctionnement

Phase de recherche – Stockage des données



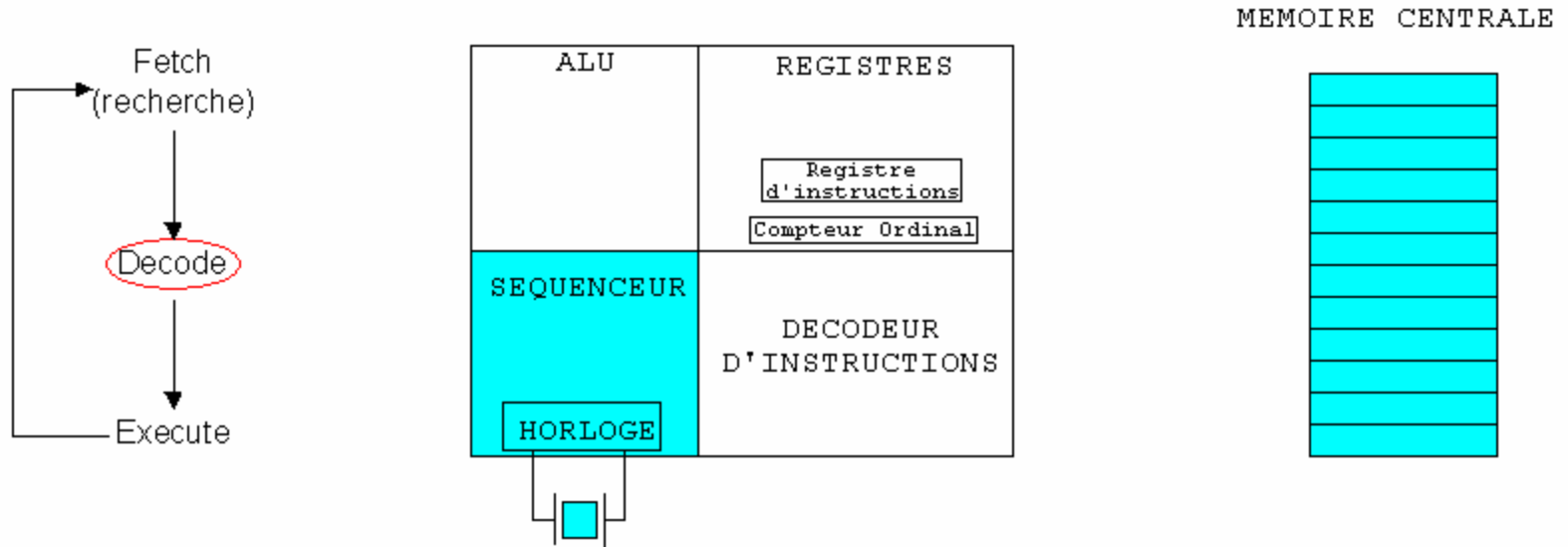
Le CPU stocke la donnée dans le registre d'instructions





Description du fonctionnement

Phase de décodage



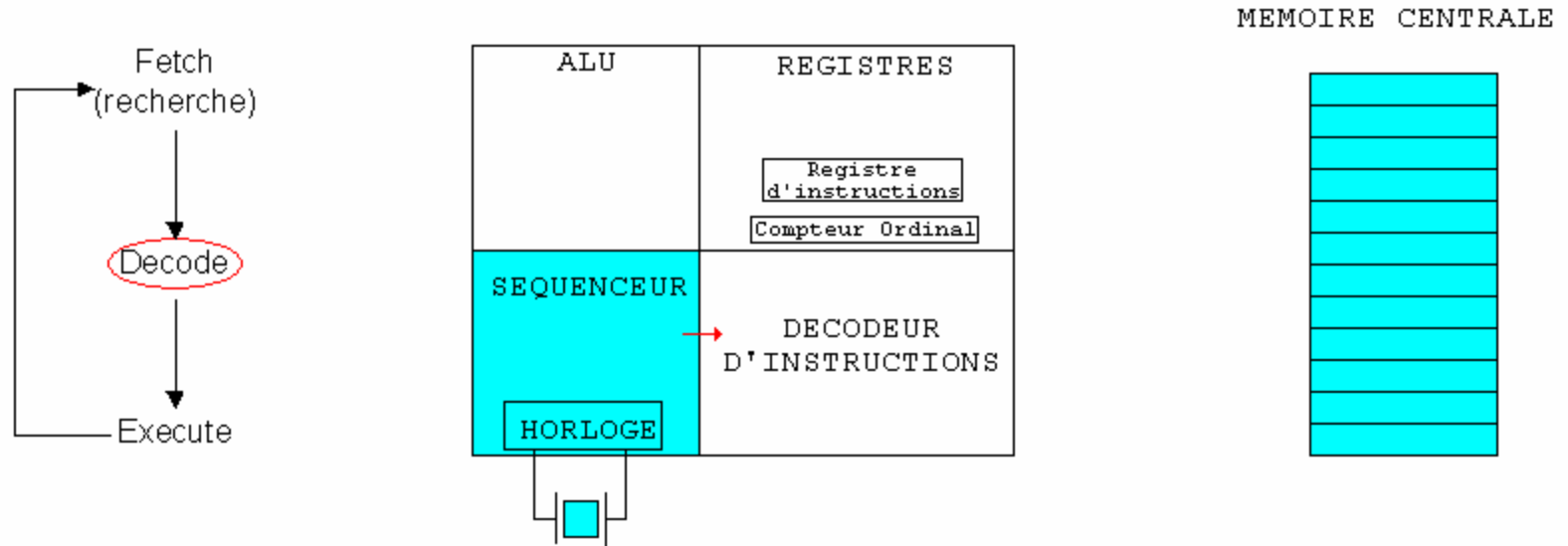
On passe à la phase décodage de l'instruction (quels organes du CPU vont être mis en œuvre en fonction de la valeur stockée dans le registre d'instructions)





Description du fonctionnement

Phase de décodage – Décodage de l'instruction



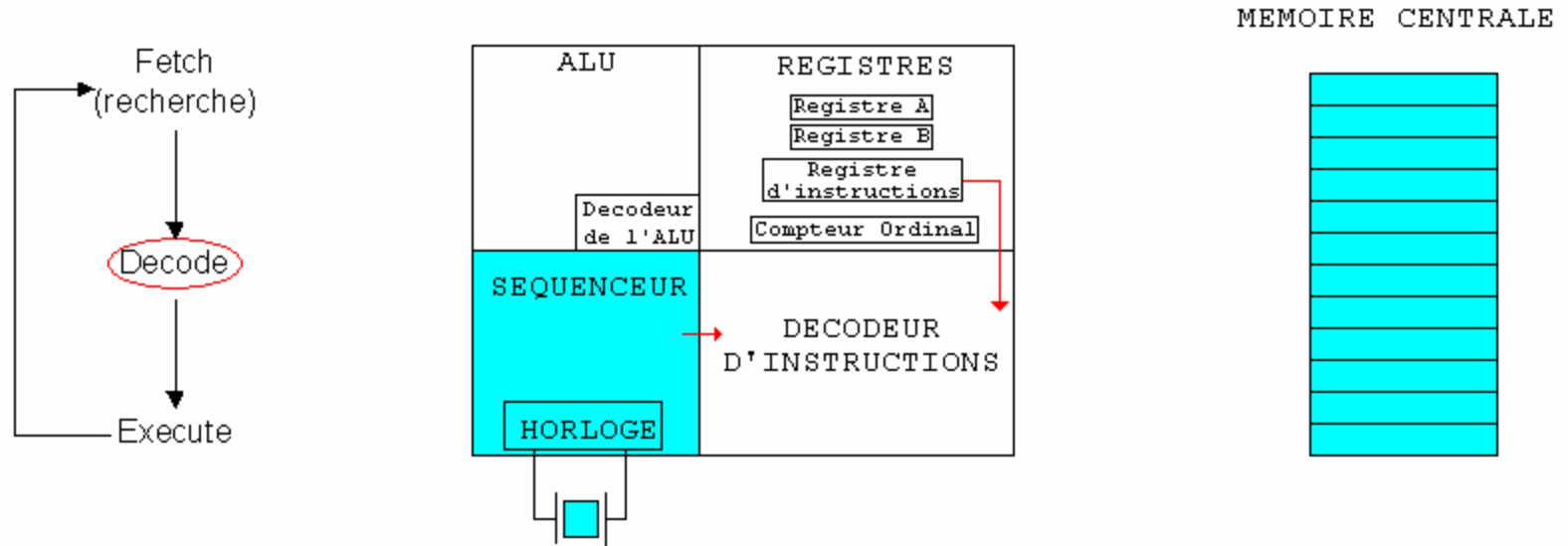
Le séquenceur active le décodeur d'instructions





Description du fonctionnement

Phase de décodage – Décodage de l'instruction



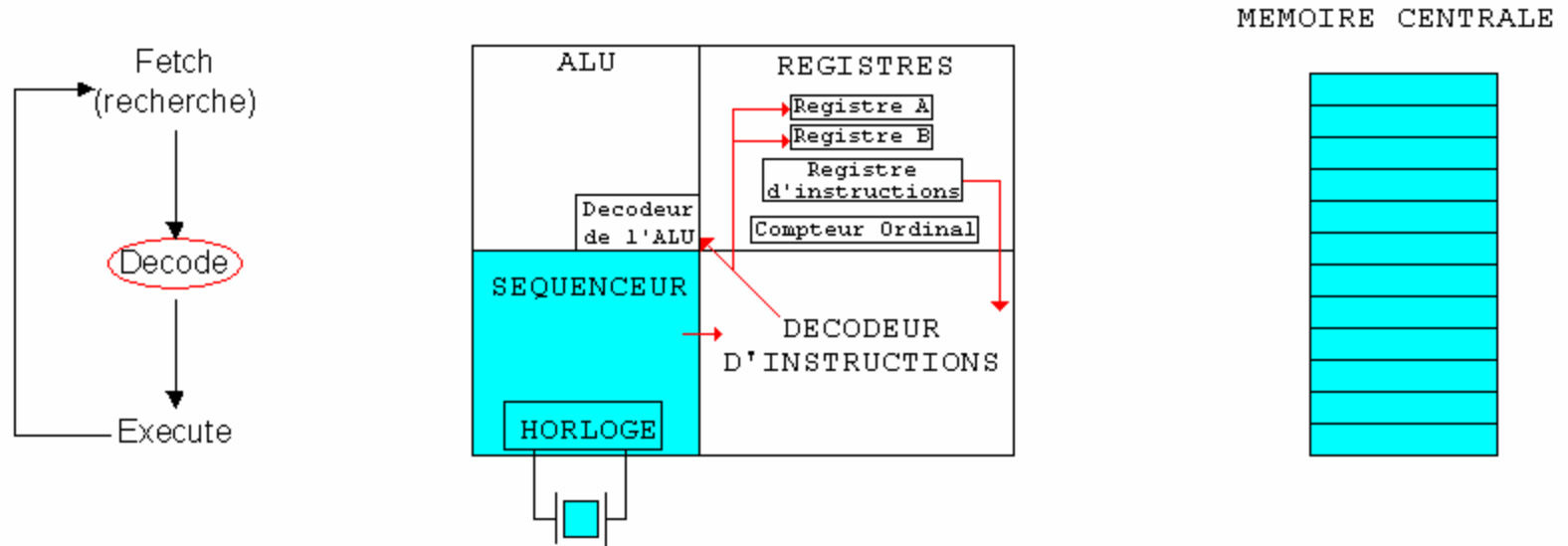
Le décodeur d'instruction lit le contenu du registre d'instructions.





Description du fonctionnement

Phase de décodage – Pré-activation des éléments du CPU



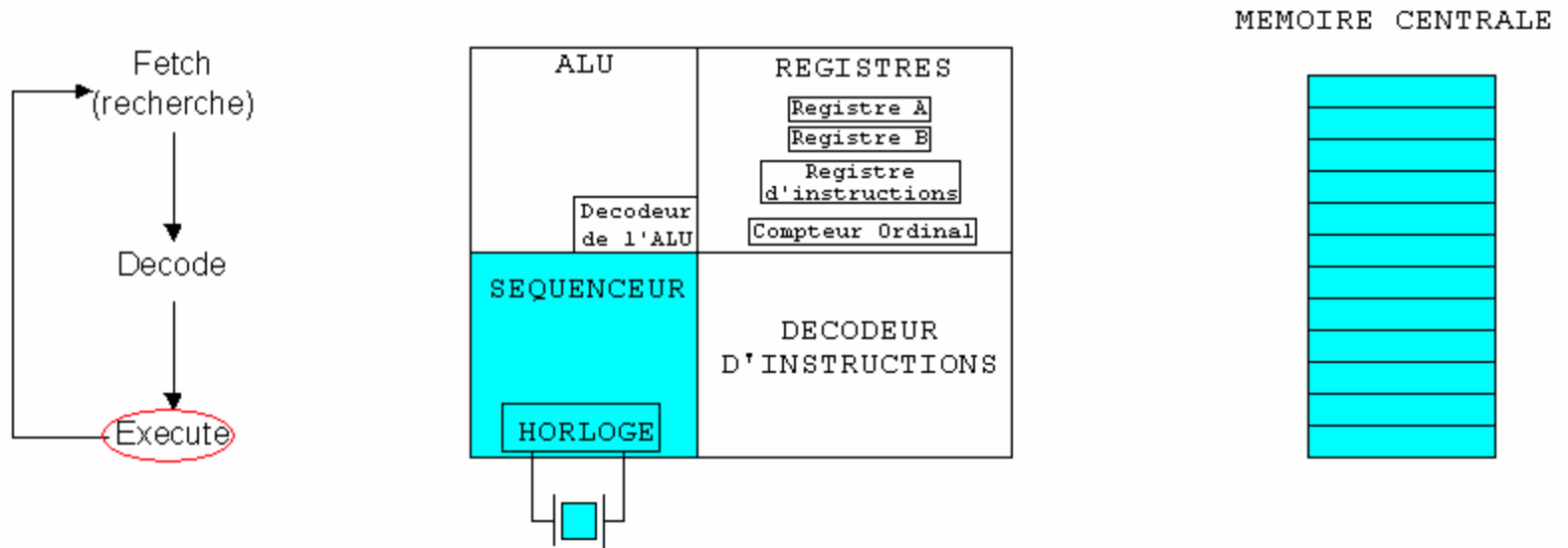
Le décodeur d'instruction pré-active les éléments correspondant : il envoie un code vers le décodeur de l'ALU et il sélectionne les registres qui vont être utilisés en entrée et en sortie.





Description du fonctionnement

Phase d'exécution



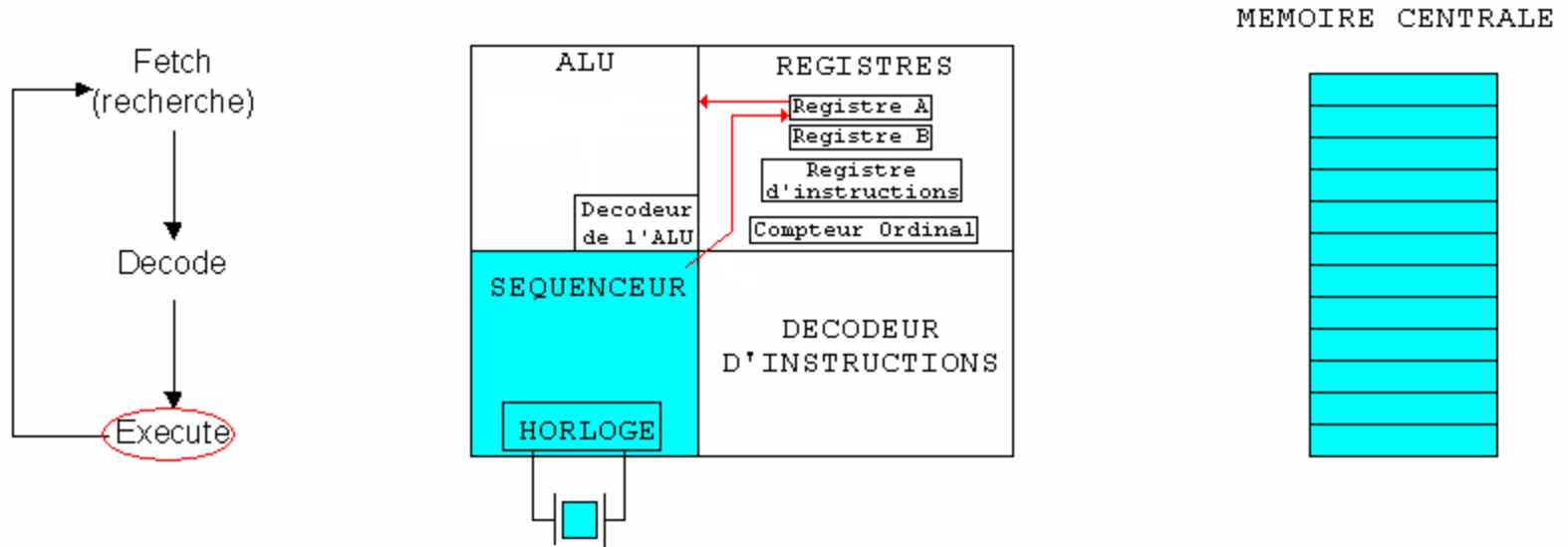
On passe à la phase d'exécution





Description du fonctionnement

Phase d'exécution – Lecture du registre sélectionné



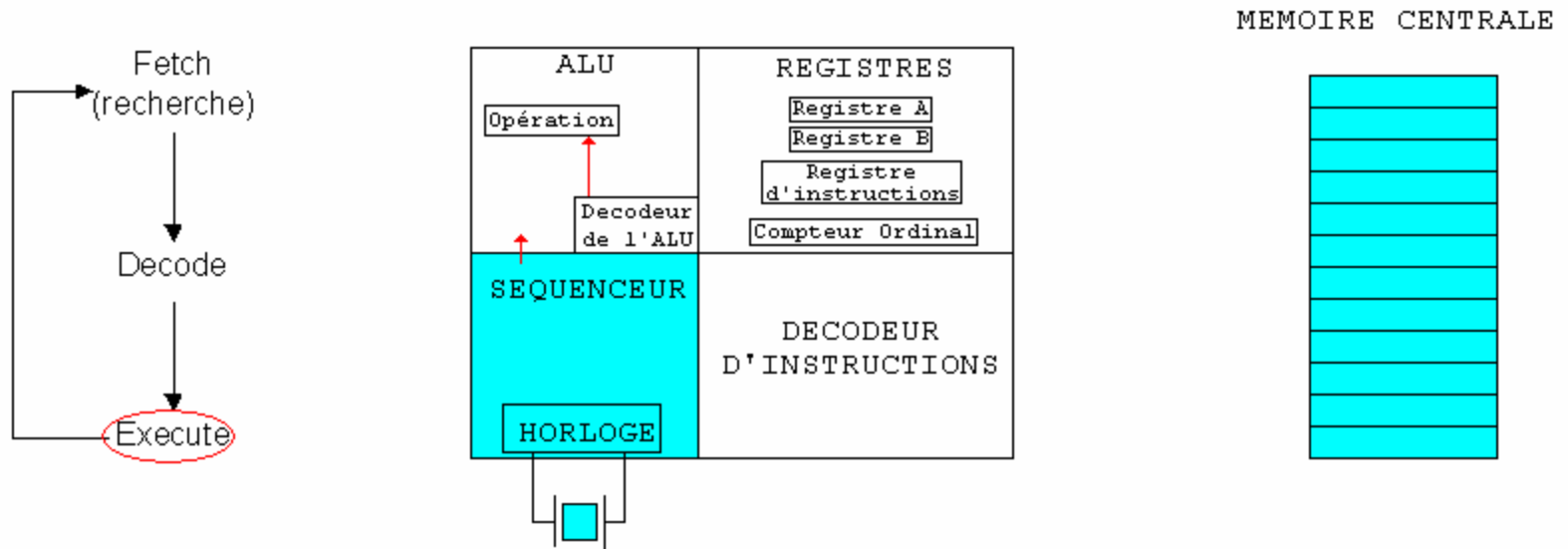
L'opérateur lit les données sur lesquels doivent être effectués des traitements depuis les registres qui ont été sélectionnés auparavant.





Description du fonctionnement

Phase d'exécution – Activation des éléments de l'ALU



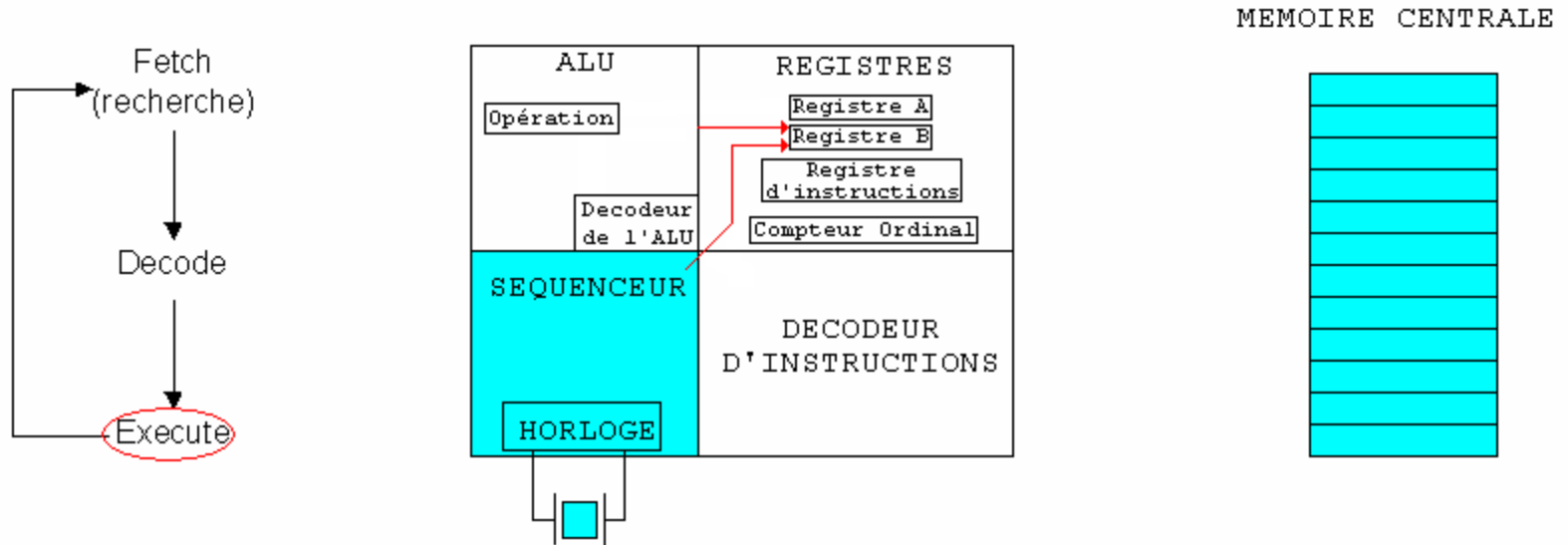
Le décodeur de l'ALU active l'opérateur sélectionné





Description du fonctionnement

Phase d'exécution – Ecriture dans les registres sélectionnés



L'opérateur écrit les résultats de l'opération dans les registres qui ont été sélectionnés auparavant.



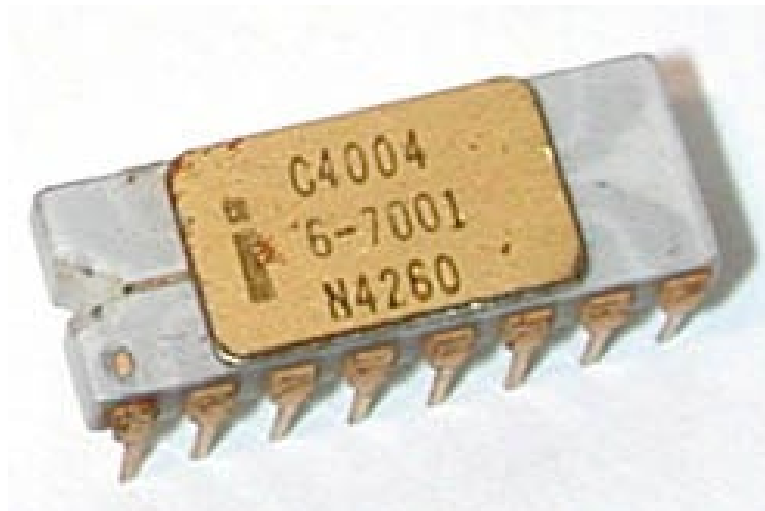


Le cas 4004

Présentation historique

Après avoir vu le fonctionnement d'un processeur sur un schéma de principe, nous allons voir le fonctionnement d'un processeur réel – le 4004 – sur son schéma bloc.

Ce processeur, développé en 1971, est le **premier microprocesseur**.





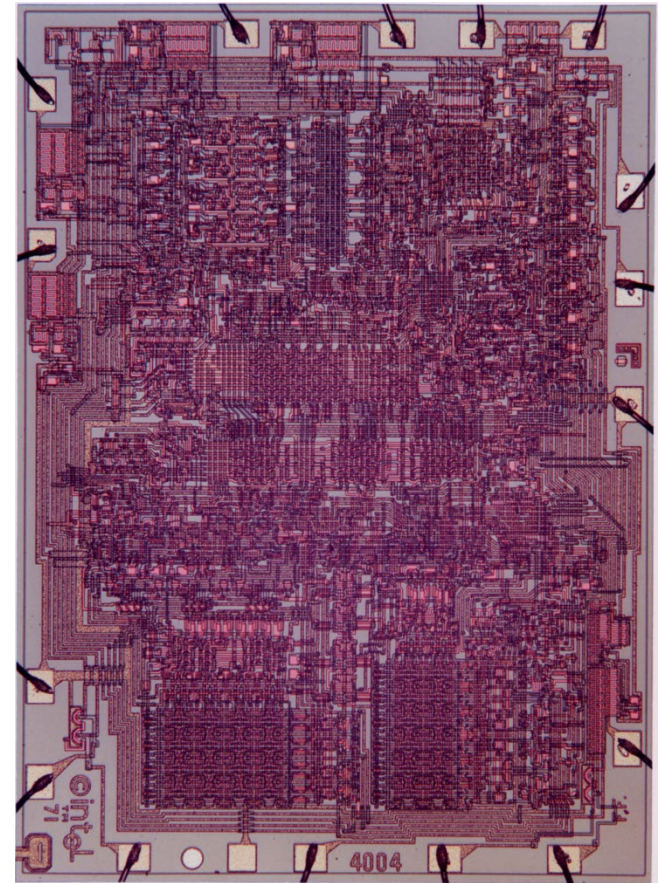
Le cas 4004

Présentation historique

C'était le premier circuit intégré incorporant tous les éléments d'un ordinateur dans un seul boîtier :

- unité de calcul,
- mémoire,
- contrôle des entrées/sorties.

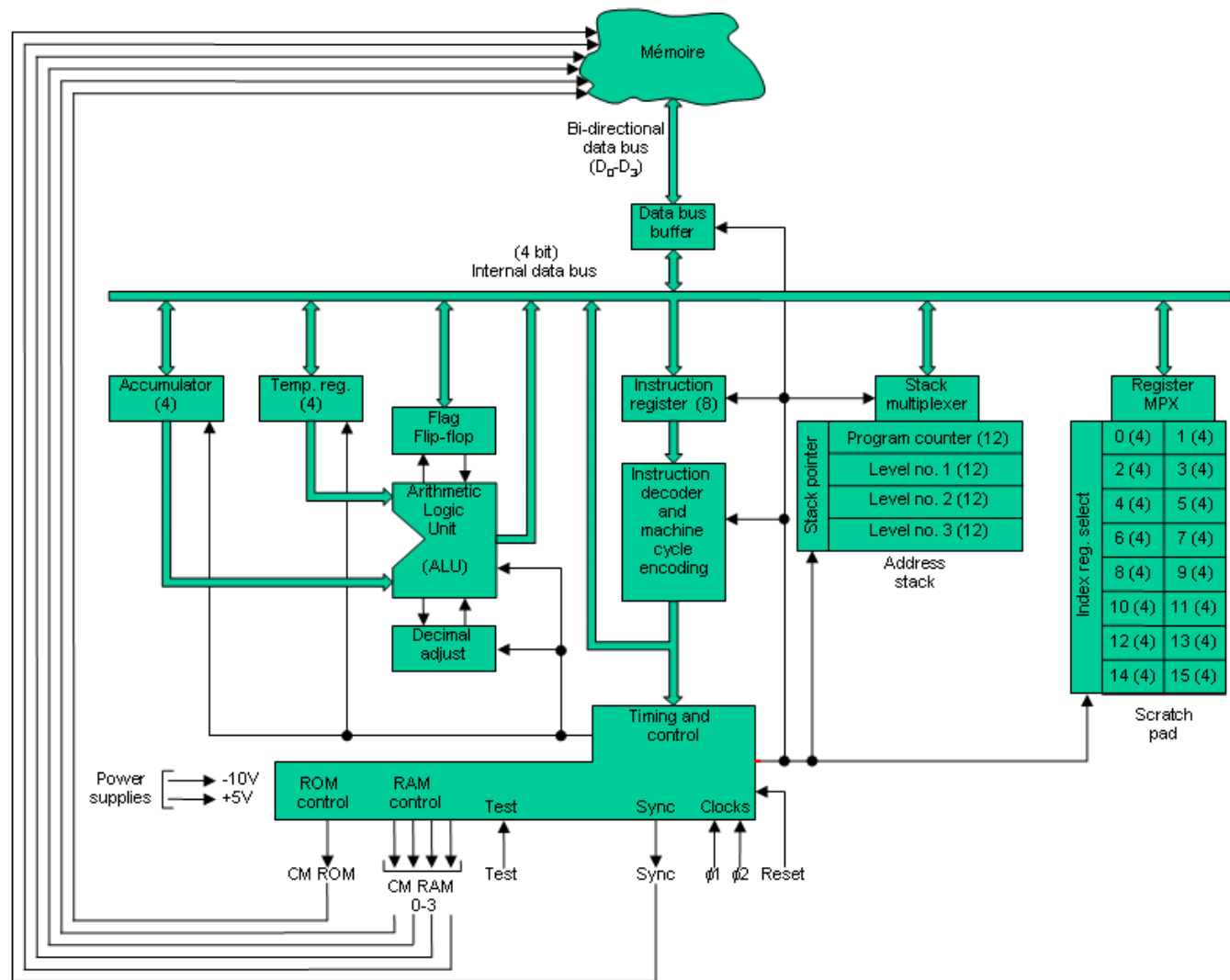
Auparavant, il fallait plusieurs circuits intégrés différents, chacun dédié à une tâche particulière.





Le cas 4004

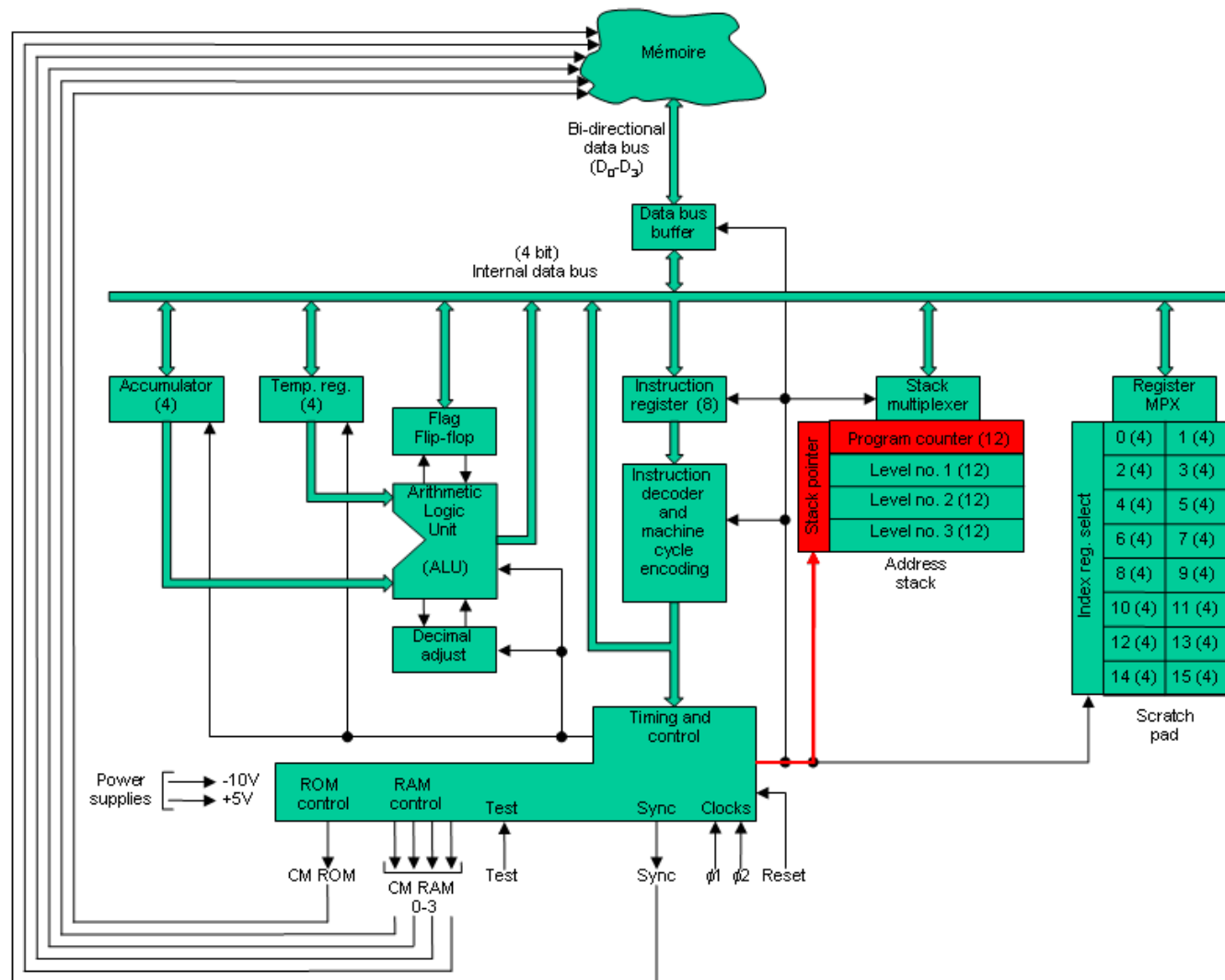
Le schéma bloc interne du 4004





Le cas 4004

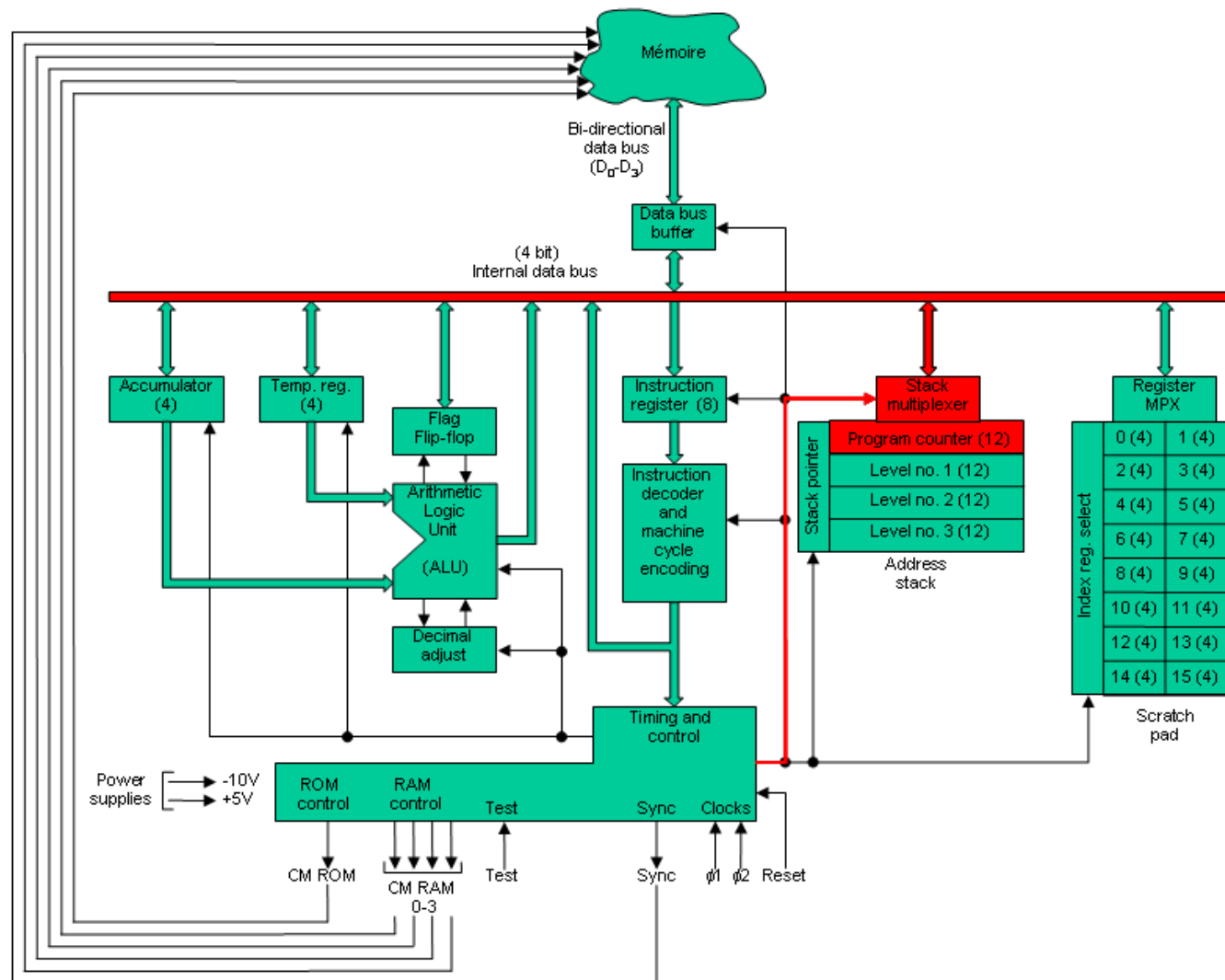
Recherche : Incrémentation du compteur ordinal





Le cas 4004

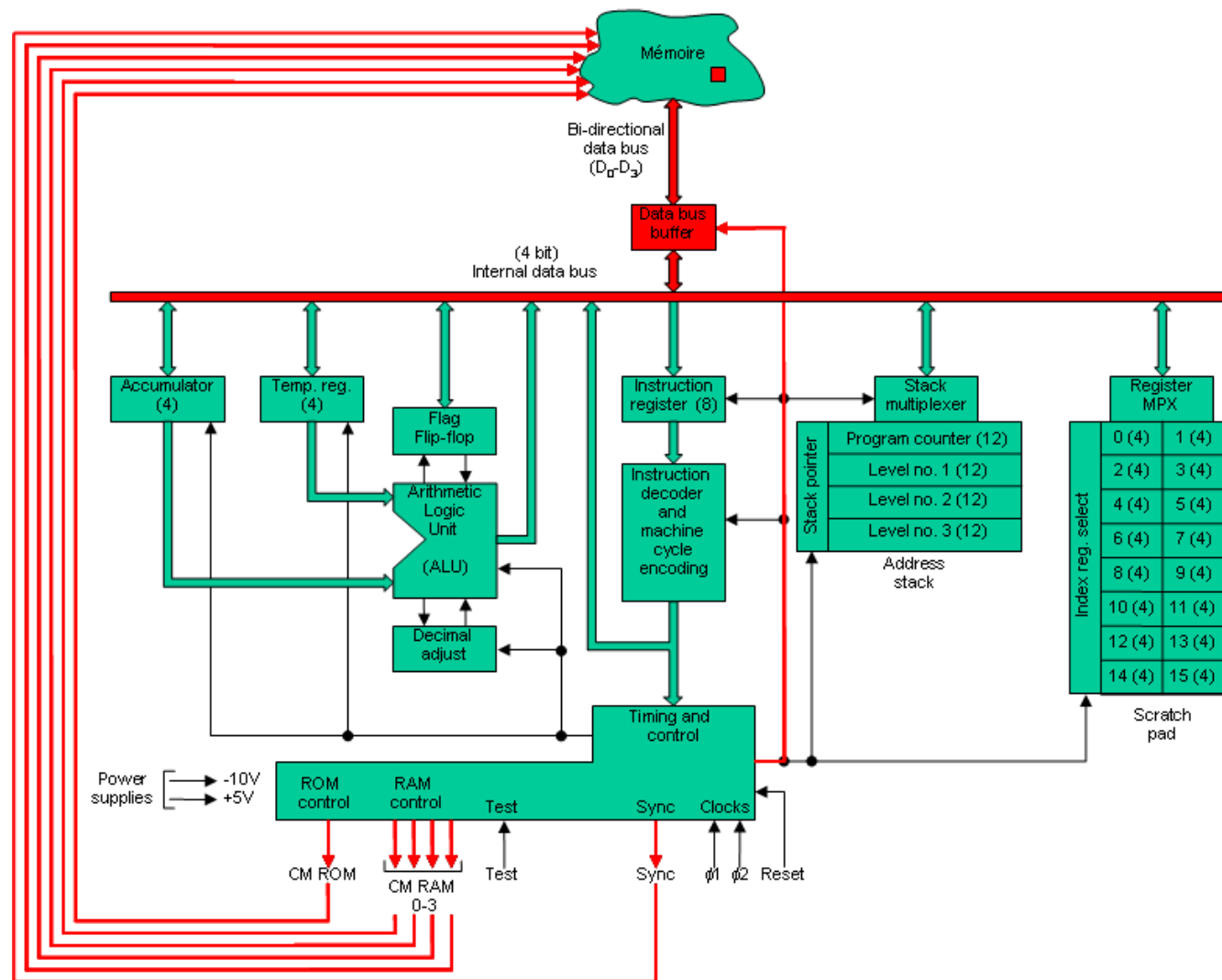
Recherche : Adresse présente sur le bus d'adresse interne





Le cas 4004

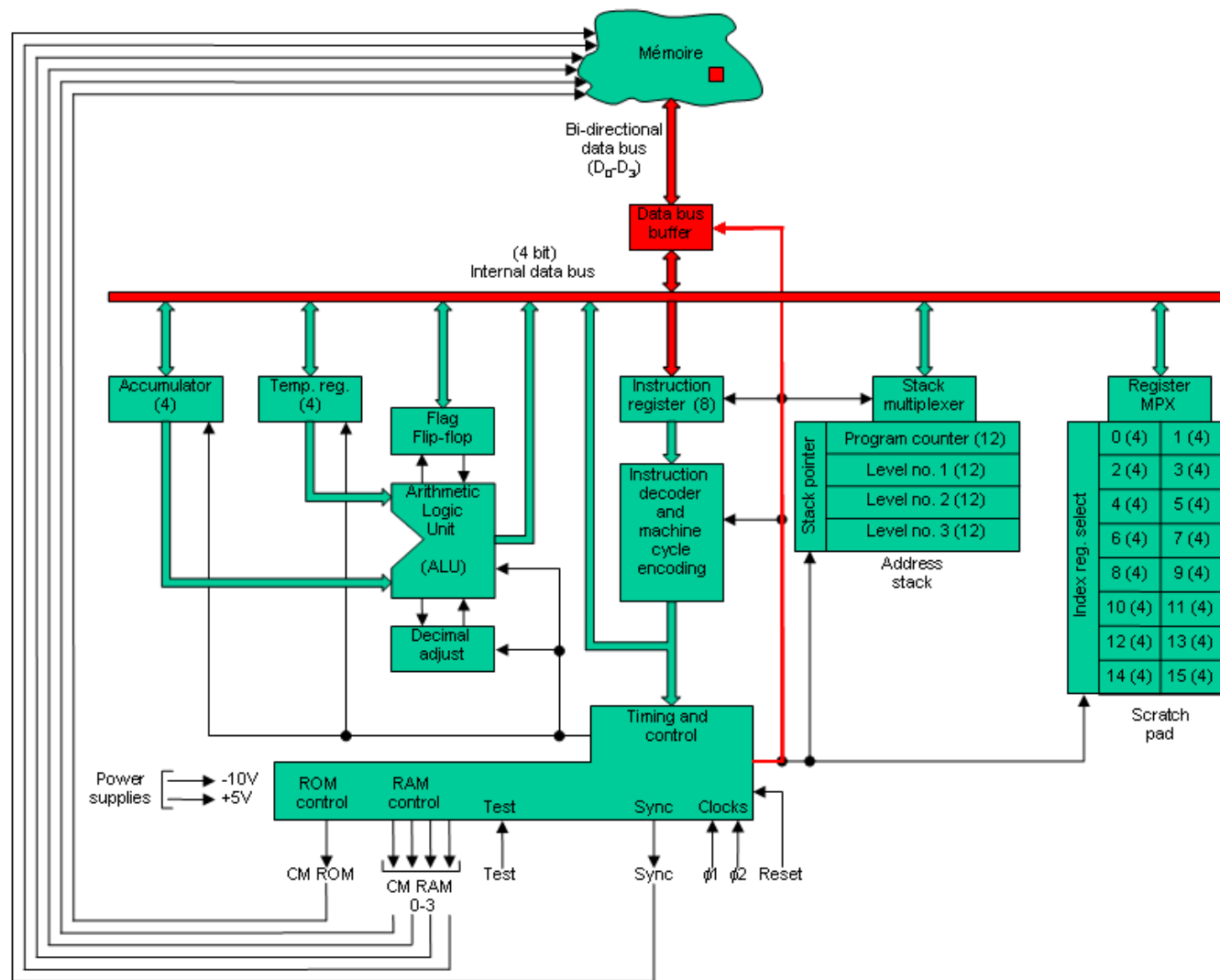
Recherche : Sélection de la case mémoire





Le cas 4004

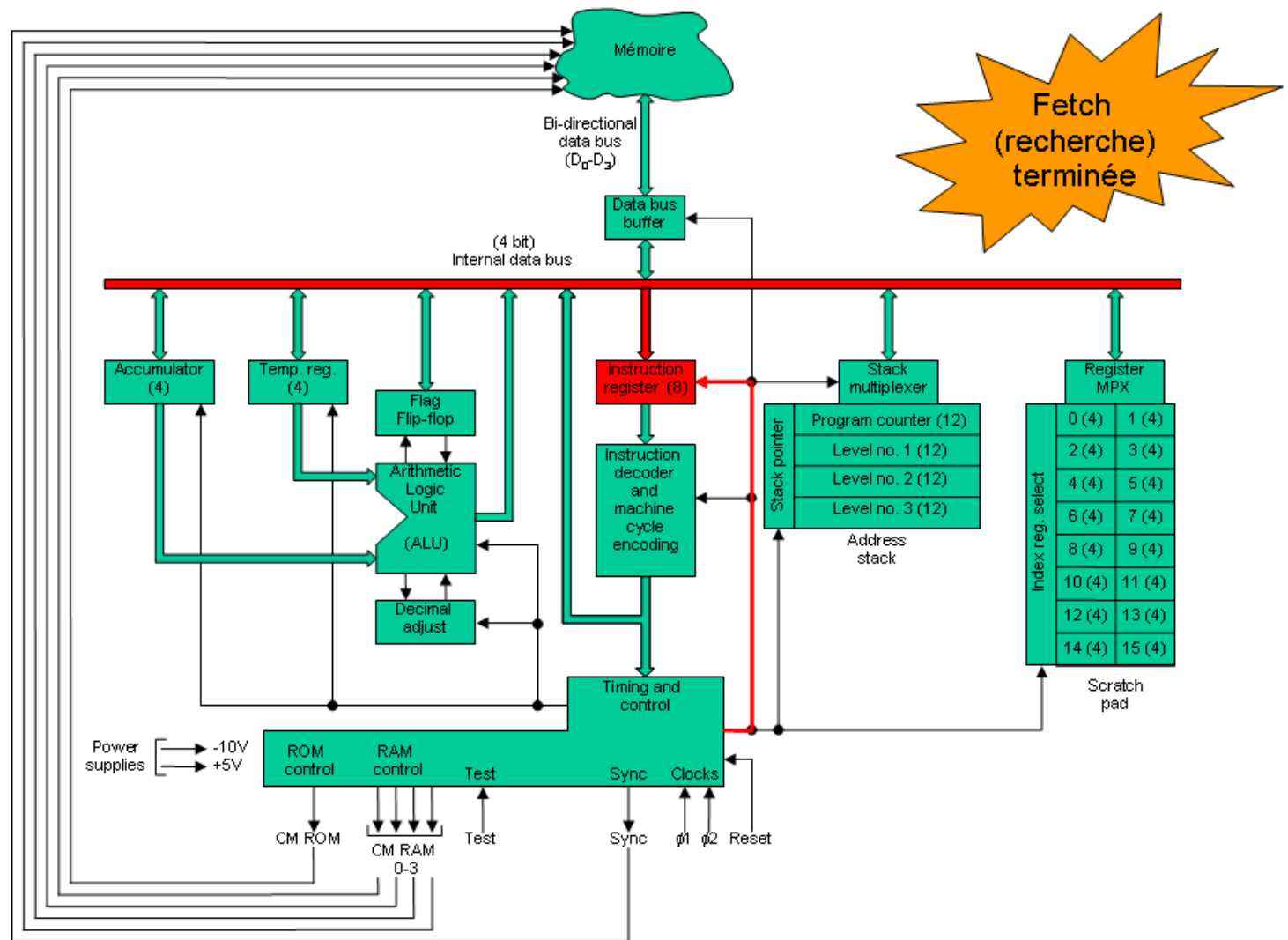
Recherche : Instruction transmise vers le processeur





Le cas 4004

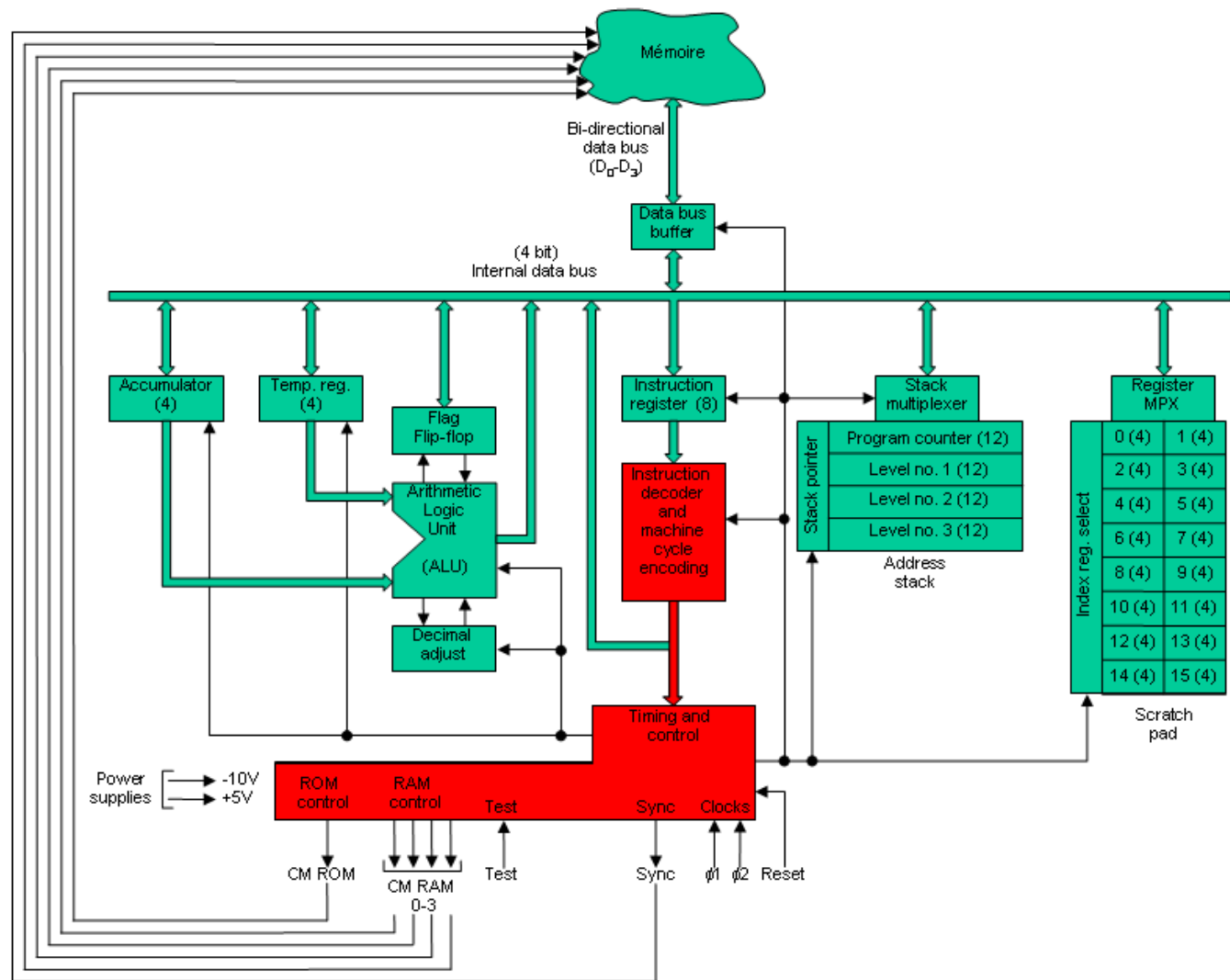
Recherche : Instruction stockée dans le registre





Le cas 4004

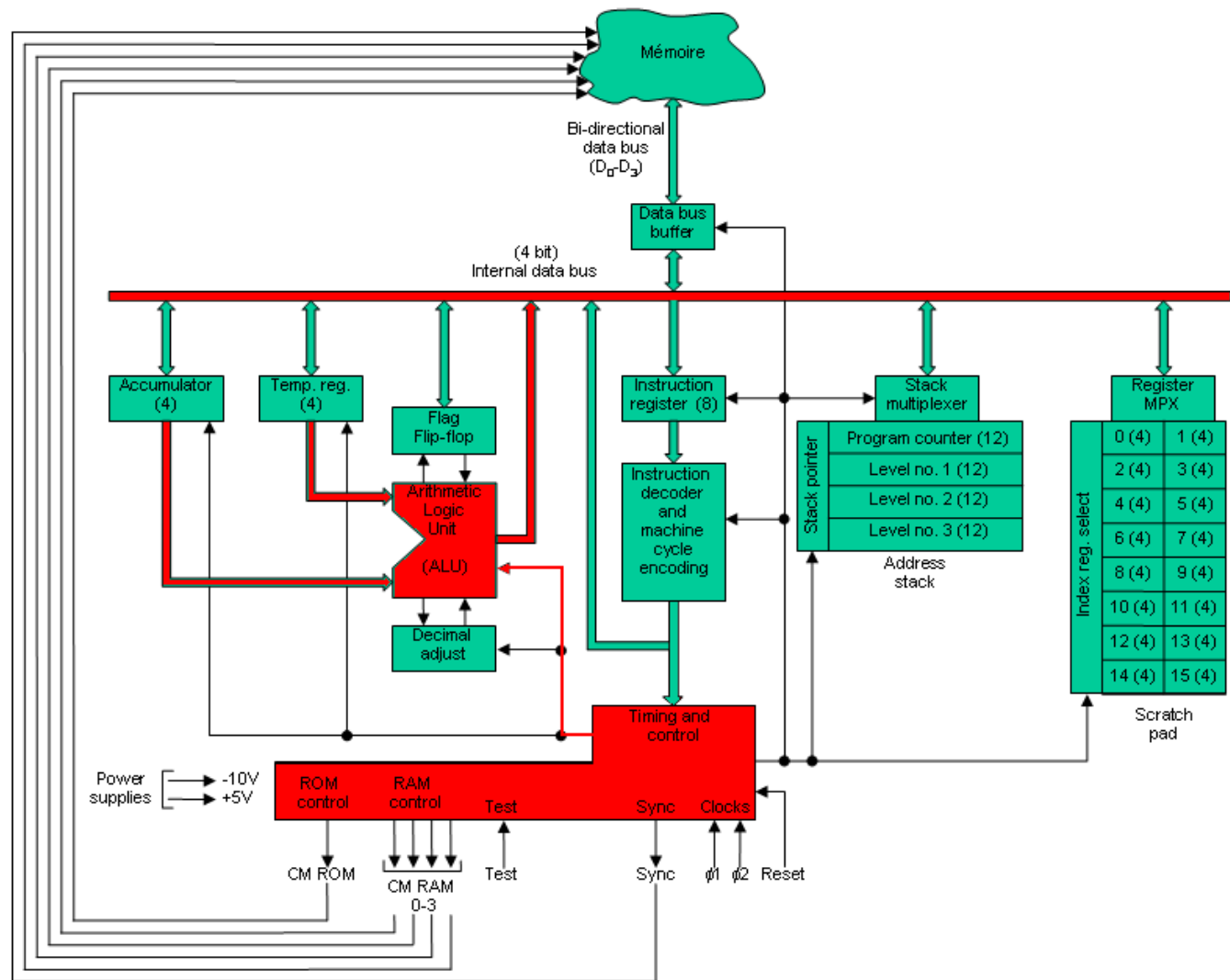
Exécution : Instruction envoyée vers le circuit de contrôle





Le cas 4004

Excution : Calcul sur des donnée préalablement stockées





Le cas 4004

Exécution : Stockage du résultat dans l'accumulateur

